AutonoStack

Propuesta de Proyecto Intermodular

—

Lucía Fernández Florencio

DAW2A

1.1. Introducción y Propósito del Documento..... 4
1.2. Contexto, Visión y Justificación..... 4
1.3. Objetivos del Proyecto..... 5
2.1. El Reto: Conceptualización y Filosofía de AutonoStack..... 6
2.2. Anatomía del Logotipo y Tipografía..... 7
2.3. Justificación Técnica y Funcional de la Paleta Cromática..... 8
2.4. Proceso de Diseño y Adaptabilidad (Responsive Branding)..... 10
2.5. Implementación Mobile-First..... 11
3.1. Fase de Consultoría y Recogida de Necesidades (Actas de Reunión)..... 12
3.2. Marco Contractual y Acuerdos de Servicio (Modelo CNPS 1.0)..... 13
3.3. Matriz de Interesados (Stakeholders) y Responsabilidades..... 15
3.4. Cumplimiento de Estándares y Normativa Vigente..... 16
3.5. Constitución del Producto Mínimo Viable (MVP)..... 16
3.6. Prospección de Mejoras Futuras y Escalabilidad Pactada..... 17
3.7. Metodología de Trabajo e Interacción..... 18
4.1. Justificación: La Necesidad de una Soberanía Financiera Real..... 18
4.2. Objetivos del Proyecto..... 20
5.1. Requisitos Funcionales (RF)..... 22
5.2. Requisitos No Funcionales (RNF)..... 29
6.1. Diagrama de Casos de Uso y Escenarios Operativos..... 35
6.2. Definición y Caracterización de los Actores del Sistema..... 35
6.3. Matriz de Responsabilidades y Acceso Técnico..... 37
6.4. Justificación Técnica y Seguridad de Acceso..... 39
6.5. Implementación y Garantía de Accesibilidad en la Gestión de Roles..... 40
7.1. Modelo Entidad-Relación (E-R)..... 41
7.2. Scripts de Creación de Estructura (DDL)..... 46
7.3. Implementación de Seguridad a Nivel de Fila (Row Level Security)..... 49
7.4. Capa de Lógica de Negocio: Algoritmos de Inteligencia Financiera..... 51
7.5. Estrategia de Persistencia y Caché de Alto Rendimiento..... 52
7.6. Lógica de Agregación: Vistas SQL para el Dashboard..... 53
7.7. Integridad Referencial y Acciones de Disparo (Triggers)..... 54
8.1. Metodología y Estrategia de Diseño UX: La Neuroergonomía del Control..... 56
Tabla 8.1: Criterios de Excelencia en el Diseño UX..... 56
8.2. El Corazón Analítico: El Dashboard como Centro de Inteligencia Visual..... 57
8.3. Iconografía: Semántica Visual y Claridad Simbólica..... 58
Tabla 8.2: Familia de Iconos y Justificación..... 59
8.4. Desglose de Pantallas Principales: Ingeniería de Vistas..... 60
8.5. Diseño de Interfaz (UI) y Semiótica del Color..... 61
Tabla 8.3: Semiótica Cromática Aplicada a la Gestión Financiera..... 61
8.6. Ingeniería de Accesibilidad (Compromiso WCAG 2.1 AA)..... 62
8.7. Paradigma Adaptativo: Universalidad Mobile-First..... 62

Tabla 8.4: Comportamiento Responsive de los Componentes.....63
8.8. Tabla Resumen de Pantallas Principales del Ecosistema..... 63
9.1. Arquitectura General del Sistema..... 65
9.2. Tecnologías de Frontend (Capa de Presentación e Interacción).....65
Tremor: La Librería de Dashboards para Consultoría..... 67
Recharts: Visualización Componible y Reactiva.....68
Matriz de Justificación Técnica: Tremor vs. Recharts.....68
Integración con la Lógica de Negocio..... 69
9.3. Gestión del Estado Global y Sincronización Asíncrona.....70
Tabla 9.3: Matriz Comparativa de Gestión de Estados..... 72
9.4. Backend e Infraestructura (Capa de Servicios)..... 72
Tabla 9.4: Desglose de Servicios de Infraestructura..... 73
Tabla 9.5: Cumplimiento de Propiedades ACID en AutonoStack.....74
9.5. Infraestructura de Despliegue y CI/CD: Vercel..... 75
9.4. Ecosistema de Desarrollo y DevOps: La Ingeniería del Ciclo de Vida..... 76
Tabla 9.6: Matriz del Ciclo de Vida de Despliegue (Pipeline)..... 77
Tabla 9.7: Matriz de Cobertura de Pruebas.....78
Tabla 9.8: Ecosistema de Blindaje del Código..... 79
9.5. Resumen de Ventajas del Ecosistema DevOps..... 80
9.5. Matriz de Justificación Técnica (Análisis Comparativo de Stack).....80
Tabla 9.8: Matriz de Decisiones Arquitectónicas y Justificación de Ingeniería.....81
9.6. Análisis Profundo de las Elecciones Estratégicas..... 83
9.7. Análisis de Eficiencia y Rendimiento Esperado (Performance Metrics)..... 84
Tabla 9.9: Objetivos de Rendimiento Técnico (SLAs)..... 85
9.8. Análisis de Costes e Infraestructura..... 86
Tabla 9.10: Desglose de Capacidades y Límites del Tier Gratuito.....86
Tabla 9.11: Estándares de Seguridad y Privacidad Integrados..... 87
Tabla 9.12: KPIs de Rendimiento de Infraestructura (SLA Proyectado)....88
10.1. Marco Metodológico: El Paradigma del Desarrollo Ágil (Agile Scrum).....89
Tabla 10.1: Pilares del Marco Ágil en AutonoStack.....90
10.2. Ciclo de Vida del Proyecto: Fases de Ingeniería.....91
10.3. Cronograma y Desglose Detallado de Actividades..... 92
10.4. Análisis de la Ruta Crítica (Critical Path Method).....94
10.5. Gestión de Tareas y Flujo de Trabajo (Workflow).....95
10.7. Matriz Resumen de Hitos Críticos..... 95
10.8. Justificación de la Planificación..... 96
10.9. Gestión de Riesgos y Mitigación Estratégica..... 97
Tabla 10.3: Matriz de Gestión de Riesgos Tecnológicos..... 97
10.10. Ecosistema de Herramientas de Planificación y Control..... 98
10.11. Resumen de la Lógica de Negocio e Inteligencia Estratégica.....98

10.12. Gestión de Recursos y Capacidades..... 98
10.13. Aseguramiento de la Calidad y Control (QA).....99
10.9. Gestión de Riesgos y Planes de Contingencia.....100
10.9.1. Matriz de Riesgos y Protocolos de Respuesta..... 100
10.10. Planes de Contingencia Operativos..... 101
10.11. Gestión de Recursos y Garantía de Calidad (QA).....102
11.1. Arquitectura del Sistema..... 103
11.2. Entorno de Desarrollo y Requisitos..... 103
11.3. Diccionario de Datos y Modelo Relacional..... 104
11.3.1. Especificación de Tablas Principales..... 104
11.4. Seguridad y Row Level Security (RLS).....105
11.5. Mantenimiento y Resolución de Problemas (Troubleshooting).....106
11.6. Interfaz de Programación de Aplicaciones (API) y Servicios Externos.....106
11.7. Guía de Despliegue y Configuración en la Nube (Vercel)..... 107
11.8. Protocolos de Mantenimiento y Cierre de Proyecto.....109
11.9. Organización del Código Fuente y Modularidad.....109
11.9.1. Árbol de Directorios (/src).....109
11.10. Lógica de Componentes y Hooks Personalizados.....110
11.11. Procedimientos de Pruebas y Aseguramiento de Calidad (QA).....111
11.12. Gestión de Dependencias y Seguridad de Paquetes.....111
11.13. Consideraciones Finales de Ingeniería..... 112
12.1. Cumplimiento de Objetivos y Hitos..... 113
12.2. Valoración Personal y Aprendizaje..... 114
Anexo I: Matriz de Trazabilidad de Requisitos.....115
Anexo II: Glosario Técnico..... 118

1. Presentación del Proyecto y Contexto Estratégico

1.1. Introducción y Propósito del Documento

Este informe técnico detalla de forma pormenorizada la arquitectura, el diseño y la implementación de **AutonoStack**, una plataforma digital concebida en el marco del módulo de «Proyecto Intermodular». La finalidad de este documento es servir como guía integral del ciclo de vida del software, abarcando desde el análisis de necesidades y el marco jurídico inicial hasta el despliegue final en un entorno de producción real.

A través de sus apartados, se justifica la viabilidad del proyecto y se demuestra la solvencia técnica en el uso de tecnologías modernas como **React** para el desarrollo de interfaces y **Supabase** para la gestión de servicios de infraestructura (BaaS), consolidando así un ecosistema Full Stack robusto y escalable.

1.2. Contexto, Visión y Justificación

En un entorno económico donde el trabajo por cuenta propia gana terreno, la gestión administrativa se ha convertido en un obstáculo para la productividad del autónomo. Tras evaluar el mercado actual, se observa que muchas soluciones fallan al ser demasiado genéricas o carecer de un componente analítico que ayude al profesional a entender su rentabilidad real.

AutonoStack surge para cubrir este vacío, ofreciendo una visión de **soberanía financiera**. La propuesta no se limita a un mero volcado de tickets y facturas; su valor diferencial reside en la capacidad de cruzar datos entre ingresos, gastos deducibles y proyectos específicos. El objetivo es que el usuario deje de percibir la contabilidad como una carga burocrática y empiece a utilizarla como una

herramienta estratégica para optimizar sus beneficios y prever sus obligaciones fiscales con total claridad.

1.3. Objetivos del Proyecto

El desarrollo de la plataforma se rige por un conjunto de metas diseñadas para garantizar una herramienta profesional y de alta disponibilidad:

- **Objetivo General:** Desarrollar un ecosistema web integral que centralice la gestión financiera del profesional independiente, transformando los datos operativos en indicadores de rendimiento de fácil interpretación.
- **Objetivos Específicos:**
    - **Seguridad y Acceso:** Establecer un control de acceso perimetral mediante **Supabase Auth**, implementando niveles de visibilidad diferenciados para administradores, usuarios activos y perfiles de demostración.
    - **Gestión Operativa:** Construir módulos dinámicos para el mantenimiento (CRUD) de la facturación emitida y recibida, asegurando la correcta categorización de los gastos sujetos a desgravación.
    - **Trazabilidad por Proyectos:** Implementar un sistema de asociación lógica que permita vincular cada movimiento financiero a un proyecto concreto, facilitando el cálculo del margen neto de cada actividad.
    - **Inteligencia de Datos:** Integrar paneles de visualización con **Tremor / Recharts** para monitorizar en tiempo real el flujo de caja y generar previsiones automáticas del IVA trimestral.
    - **Usabilidad y Adaptabilidad:** Diseñar una interfaz intuitiva con **Tailwind CSS** que garantice una navegación fluida y consistente, independientemente del dispositivo de acceso.

- **Persistencia Segura:** Garantizar la integridad de la información mediante el uso de **PostgreSQL** y la aplicación de reglas de seguridad a nivel de registro (**RLS**) para proteger la privacidad del usuario.

2. Imagen Corporativa y Filosofía de Marca

2.1. El Reto: Conceptualización y Filosofía de AutonoStack

AutonoStack nace no solo como una herramienta técnica, sino como un manifiesto de **independencia y claridad financiera** para el trabajador por cuenta propia. Nuestra filosofía se fundamenta en el concepto de **"Arquitectura de Control"**: la convicción de que un autónomo no debe ser un esclavo de su contabilidad, sino el arquitecto jefe de su propia rentabilidad. En un sector saturado de aplicaciones bancarias genéricas o softwares de gestión excesivamente burocráticos, AutonoStack se posiciona como el aliado estratégico que elimina el ruido informativo para priorizar la salud económica real.

El desafío fundamental al construir esta marca consistió en proyectar una identidad visual que inspirara **confianza institucional** y, simultáneamente, **agilidad operativa**. La marca debe transmitir que, tras una interfaz moderna y fluida, reside un motor de cálculo robusto capaz de gestionar previsiones de IVA, desgravaciones complejas y balances de proyectos con una precisión matemática absoluta. AutonoStack es la respuesta a la fragmentación financiera; es el lugar donde el caos de los tickets y las facturas se transforma en una pila (*stack*) ordenada de datos accionables.

2.2. Anatomía del Logotipo y Tipografía

El sistema visual de **AutonoStack** ha sido desarrollado bajo un riguroso esquema de proporciones matemáticas que evoca orden, estructura y crecimiento sostenido. Cada elemento del logotipo ha sido diseñado para funcionar con la máxima legibilidad, tanto en la cabecera de un dashboard interactivo como en una interfaz de gestión móvil.

- **Imagotipo:** El símbolo de AutonoStack representa una **gráfica de barras ascendente** integrada con una flecha dinámica que apunta hacia el ángulo superior derecho. Esta disposición simboliza la acumulación de capital y la solidez de los datos financieros que el usuario construye día a día. La flecha naranja refuerza visualmente la idea de **escalabilidad, rentabilidad y progreso económico**. El conjunto reposa sobre una línea de base arqueada que sugiere un horizonte de estabilidad y transparencia total sobre cada factura y gasto registrado.

- **Logotipo (Wordmark):** El nombre de la marca se presenta mediante una tipografía moderna con un degradado que transita del verde esperanza al azul corporativo. Esta transición cromática simboliza la evolución desde el registro de ingresos brutos hacia la consolidación de un balance neto seguro y profesional.

- **Tipografía:** Se ha seleccionado la familia tipográfica **Inter** como pilar de la comunicación visual del proyecto. Esta fuente Sans Serif de corte tecnológico ha sido escogida tras exhaustivas pruebas de rendimiento en interfaces de visualización de datos financieros.
    - **Claridad en cifras:** Inter posee un diseño de glifos optimizado que permite distinguir con total precisión números críticos, algo vital en una herramienta donde la precisión en el registro de facturas es innegociable.
    - **Ergonomía visual:** El interletrado natural de la fuente y su generosa altura de X permiten que el usuario navegue por tablas de gastos desgravables densas sin sufrir fatiga visual.
    - **Jerarquía de información:** Mediante el uso estratégico de pesos tipográficos, la fuente guía el ojo del autónomo hacia la información más urgente de su dashboard, como las previsiones de IVA o el beneficio neto por proyecto.

2.3. Justificación Técnica y Funcional de la Paleta Cromática

La selección de la gama de colores para **AutonoStack** trasciende la estética para convertirse en una herramienta de usabilidad diseñada para optimizar la interpretación de datos financieros complejos. Cada tono presente en el logotipo y en la interfaz de usuario ha sido seleccionado bajo criterios de funcionalidad técnica, buscando que el profesional autónomo identifique de forma instantánea el estado de su liquidez, sus obligaciones fiscales y la rentabilidad de sus proyectos.

A continuación, se detallan las razones técnicas por las que esta paleta cromática es la solución óptima para el ecosistema de la plataforma:

- **Verde Esperanza / Digital (#4ADE80) - Identificador de Flujo Positivo:** Este color se emplea como el indicador primario de **ingresos, facturas cobradas y saldos positivos**. La razón de su uso radica en la convención universal de los sistemas contables donde el verde señala la salud financiera, permitiendo que, al abrir el dashboard, el usuario perciba de un vistazo su volumen de facturación sin necesidad de leer etiquetas de texto.

- **Azul Corporativo / Seguridad (#1D4ED8) - Estructura y Rigor Técnico:** El azul se utiliza para los elementos de control, navegación y **gestión de datos sensibles**. La justificación técnica de este tono es su alta estabilidad visual y su asociación con entornos de desarrollo de software y banca moderna. Al ser un color que no fatiga la vista, es ideal para las tablas de registros de facturas y el panel de administración donde el usuario pasa la mayor parte del tiempo operativo.

- **Naranja Dinámico (#F59E0B) - Alerta de Gestión y Previsión Fiscal:** La flecha naranja del logotipo y su uso en la interfaz responden a la necesidad de resaltar **hitos críticos y previsiones de IVA**. Se ha elegido este color frente al rojo para evitar una respuesta de estrés innecesaria en el autónomo; el naranja actúa como una "alerta constructiva" que señala pagos pendientes o cierres de trimestre, manteniendo una estética de proactividad y crecimiento.

- **Contraste y Accesibilidad (Gris/Blanco) - Cumplimiento normativo WCAG:** El uso de fondos neutros y textos en gris oscuro no es accidental, sino que responde a la necesidad de alcanzar un nivel de conformidad **AA en accesibilidad**. Esto garantiza que los indicadores cromáticos de las gráficas (verdes, azules y naranjas) resalten sobre el fondo, asegurando que cualquier usuario, independientemente de sus capacidades visuales, pueda distinguir correctamente entre un gasto desgravable y un ingreso neto.

2.4. Proceso de Diseño y Adaptabilidad (Responsive Branding)

La identidad visual de **AutonoStack** ha sido concebida bajo el paradigma de **"Universalidad Adaptativa"**. En un entorno donde el profesional autónomo alterna constantemente entre la gestión en movilidad (Smartphone) y el análisis profundo en oficina (Desktop), la marca debe mantener una integridad absoluta sin sacrificar la legibilidad de los datos financieros en ningún momento. El proceso de diseño no se limitó a la creación de una imagen estática, sino al desarrollo de un ecosistema visual capaz de reconfigurarse dinámicamente para servir a su propósito principal: la claridad informativa.

2.4.1. Compromiso con la Accesibilidad de Alto Nivel (WCAG AA/AAA)

El pilar fundamental de AutonoStack es la inclusión digital. Siguiendo las pautas de la **W3C (WCAG 2.1)**, la marca y su interfaz han sido sometidas a rigurosos controles de contraste y percepción:

- **Contraste Lumínico:** Se ha verificado que la relación de contraste entre los textos (gris antracita y azul corporativo) y el fondo siempre supere el ratio de **4.5:1**, alcanzando en las métricas críticas de saldos y beneficios un nivel **AAA (7:1)**. Esto asegura que usuarios con visión reducida o en condiciones de luz solar directa (uso en exteriores) puedan leer sus finanzas sin dificultad.
- **Identificación No Cromática:** AutonoStack no confía exclusivamente en el color para transmitir estados. Aunque el verde indica éxito y el naranja alerta, cada indicador visual en el dashboard va acompañado de **iconografía descriptiva de la familia Lucide** y etiquetas de texto. Así, un usuario con daltonismo puede distinguir entre un ingreso y un gasto no solo por el tono, sino por la dirección de la flecha y el símbolo asociado.
- **Escalabilidad de Tipografía (Responsive Typography):** Mediante el uso de unidades relativas en CSS (rem/em) y la familia tipográfica **Inter**, el sistema

permite el escalado del texto hasta un **200%** sin que la interfaz se rompa o pierda funcionalidad. Esto es vital para la accesibilidad móvil, donde el usuario puede necesitar ampliar la vista de una factura específica.

- **Navegación por Teclado y Foco Visual:** Todos los elementos interactivos de la marca (botones, filtros y selectores de proyecto) cuentan con un estado de **foco claramente definido**. El "Foco de AutonoStack" utiliza un anillo de color azul eléctrico de alto contraste, permitiendo que usuarios que navegan mediante teclado o tecnologías asistivas sepan exactamente dónde se encuentran dentro del flujo financiero.

2.5. Implementación Mobile-First

El diseño responde a una filosofía **Mobile-First**, asegurando que la experiencia de "Soberanía Financiera" no se vea mermada en pantallas pequeñas. Los elementos del logotipo se reordenan verticalmente en dispositivos móviles, y los gráficos dinámicos de **Tremor/Recharts** se transforman en vistas de lista o tarjetas simplificadas que mantienen los mismos códigos de color y jerarquía visual.

3. Gestión de Consultoría y Marco Contractual

El desarrollo de **AutonoStack** trasciende la mera implementación de código para constituirse como una solución de consultoría estratégica integral. Se sustenta sobre un proceso riguroso de *discovery* y un blindaje legal diseñado para proteger la viabilidad económica y la propiedad intelectual tanto de la desarrolladora autónoma como del estudio creativo **bruma.studio**. Este capítulo detalla los acuerdos de gobernanza, la gestión de expectativas y el marco jurídico que asegura la transparencia en las entregas y la mitigación de riesgos operativos.

3.1. Fase de Consultoría y Recogida de Necesidades (Actas de Reunión)

Para garantizar que la solución tecnológica se alineara con los retos reales del sector creativo y las particularidades del trabajo por cuenta propia, se ejecutaron sesiones de consultoría enfocadas en la disección de los cuellos de botella administrativos de **bruma.studio**.

3.1.1. Acta de Reunión de Inicio y Definición Técnica (Kick-off)

DOCUMENTO: ACTA DE REUNIÓN PARA CONSOLIDAR ACUERDOS > **REF:** AR-2025-001

PROYECTO: AutonoStack | **FECHA:** 26 de septiembre de 2025

DATOS DE LA SESIÓN	DETALLE
Hora / Lugar	10:00 AM / Sede Digital bruma.studio (Reunión Telemática)
Asistente (Prestadora)	[Tu Nombre] (Desarrolladora Autónoma Full Stack)
Asistente (Cliente)	Dirección Creativa de bruma.studio

ORDEN DEL DÍA Y TEMAS TRATADOS

1. **Vaciado de Necesidades de Negocio:** Se determinó que la prioridad absoluta es la **soberanía de datos** y la capacidad de ofrecer un análisis de rentabilidad que las herramientas genéricas de facturación no cubren.

2. **Identificación de Problemáticas Críticas:** El mayor obstáculo detectado para **bruma.studio** es la dispersión de facturas y la dificultad para prever el IVA trimestral de forma anticipada, lo que afecta directamente la toma de decisiones de inversión y la salud de la tesorería.
3. **Acuerdos de Alcance (MVP):** Se definió un Producto Mínimo Viable (MVP) centrado estrictamente en el CRUD de transacciones, la gestión de proyectos individuales y un dashboard analítico interactivo con capacidad de segmentación por cliente.
4. **Compromisos de Accesibilidad e Inclusión:** Se acordó formalmente que la interfaz debe cumplir con los estándares **WCAG AA**, integrando contrastes de alto ratio y navegación semántica para garantizar la usabilidad en dispositivos móviles bajo cualquier condición lumínica.

3.2. Marco Contractual y Acuerdos de Servicio (Modelo CNPS 1.0)

La ejecución del proyecto se rige por un **Contrato de Prestación de Servicios Web**, el cual formaliza las obligaciones técnicas de la desarrolladora y los derechos de explotación de **bruma.studio**. Este documento establece un entorno de trabajo profesional y seguro basado en los siguientes pilares:

3.2.1. Cláusulas Detalladas del Contrato

- **CLÁUSULA PRIMERA: Objeto y Alcance:** La prestadora se compromete al diseño, desarrollo e implantación de la plataforma AutonoStack. Esto incluye el modelado de la base de datos relacional PostgreSQL, la programación del frontend reactivo en React y la orquestación de servicios en la infraestructura de Supabase.
- **CLÁUSULA SEGUNDA: Plazos de Ejecución y Metodología:** El proyecto se divide en fases validadas por hitos de aceptación. Se establece un

cronograma estricto que abarca desde la definición de requisitos funcionales hasta el despliegue final en producción y el traspaso oficial de credenciales.

- **CLÁUSULA TERCERA: Contraprestación Económica e Hitos de Pago:** Para garantizar la viabilidad del desarrollo y la seguridad del cliente, los abonos se estructuran de la siguiente manera:
    - **30% Inicial:** Abonado a la firma del contrato tras la sesión de *Kick-off* para el inicio de actividades de análisis.
    - **40% Desarrollo:** Condicionado a la validación de la versión Beta funcional y la estructura de la base de datos.
    - **30% Final:** Liquidación final tras el despliegue exitoso en Vercel, la auditoría de seguridad y la entrega de la documentación técnica definitiva.
- **CLÁUSULA CUARTA: Propiedad Intelectual:** Los derechos sobre el código fuente, la arquitectura de la base de datos y el diseño visual se transfieren íntegramente a **bruma.studio** tras el pago final, garantizando su total independencia tecnológica.
- **CLÁUSULA QUINTA: Confidencialidad (NDA):** Dada la naturaleza sensible de los datos financieros, se incluye una cláusula estricta de protección de información. Ningún dato real ni estrategia de precios del estudio será divulgado a terceros bajo ninguna circunstancia.

3.2.2. Garantía y Mantenimiento Evolutivo

Para asegurar la robustez de la herramienta, el marco contractual contempla:

- **Periodo de Garantía Técnica:** Un plazo de cobertura de tres meses para la corrección de errores lógicos o fallos de enrutamiento derivados del desarrollo sin coste adicional.

- **Soporte a la Accesibilidad:** Compromiso de mantener el cumplimiento normativo (WCAG AA) ante posibles actualizaciones de navegadores o dependencias críticas durante el periodo de garantía.

3.3. Matriz de Interesados (Stakeholders) y Responsabilidades

Se define un organigrama funcional para asegurar que cada fase del proyecto cuente con la supervisión adecuada y evitar conflictos de autoridad.

3.3.1. Equipo del Prestador: Desarrolladora Autónoma

Rol	**Responsable**	**Funciones Principales**
Project Manager & Lead Dev	Lucía Fernández Florencio	Gestión del cronograma, diseño del modelo relacional, codificación Full Stack y despliegue.

3.3.2. Equipo del Cliente: bruma.studio

Rol	**Perfil**	**Funciones Principales**
Product Owner	Dirección Creativa	Validación de requisitos de negocio y aprobación final de hitos y entregables.
Usuario Final	Autónomo Creativo	Operativa diaria, carga de facturación y testeo de usabilidad en entornos de producción.

3.4. Cumplimiento de Estándares y Normativa Vigente

AutonoStack se desarrolla bajo el estricto cumplimiento del marco legal español y europeo:

- **RGPD (Privacidad desde el Diseño):** Implementación de medidas técnicas como **Row Level Security (RLS)** para asegurar que cada registro financiero sea invisible para cualquier actor no autorizado.
- **Normativa de Facturación:** El sistema se diseña para seguir los estándares de integridad y legibilidad requeridos por la Agencia Tributaria, facilitando la exportación de libros de facturas coherentes para la presentación telemática de impuestos.

3.5. Constitución del Producto Mínimo Viable (MVP)

La definición del MVP para **AutonoStack** no se concibió como una versión recortada, sino como el conjunto mínimo de funcionalidades críticas que permiten a **bruma.studio** operar con seguridad jurídica y eficacia financiera desde el primer día. El núcleo del sistema se ha constituido bajo los siguientes pilares técnicos:

- **Motor de Persistencia y Seguridad (Backend):** Implementación de una base de datos relacional en **PostgreSQL** mediante la infraestructura de **Supabase**, garantizando la integridad de los datos financieros.
- **Gestión Operativa de Transacciones:** Desarrollo de módulos CRUD (Alta, Baja, Lectura y Modificación) para el registro manual de facturas emitidas, recibidas y tickets de gasto.

- **Arquitectura de Privacidad RLS:** Configuración de políticas de **Row Level Security** que aseguran que la información de cada usuario sea totalmente estanca y privada.
- **Dashboard de Control de Tesorería:** Visualización dinámica mediante gráficas de flujo de caja y un módulo de cálculo preventivo de la carga impositiva de **IVA trimestral**.
- **Gestión de Justificantes Digitales:** Capacidad de almacenamiento y vinculación de documentos (PDF/Imagen) a cada transacción para cumplir con los requerimientos de la Agencia Tributaria.

3.6. Prospección de Mejoras Futuras y Escalabilidad Pactada

A la finalización del contrato inicial y tras la entrega de la documentación técnica, se ha establecido una hoja de ruta con **bruma.studio** para futuras fases de desarrollo. Estos puntos quedan fuera del alcance del presente proyecto pero se consideran prioritarios para el crecimiento del ecosistema:

- **Fase 2: Automatización Bancaria:** Integración de APIs financieras para la conciliación automática de movimientos bancarios con las facturas registradas en el sistema, eliminando el volcado manual.
- **Fase 3: Inteligencia Predictiva de Gastos:** Implementación de algoritmos de *Machine Learning* para la categorización automática de gastos mediante el reconocimiento de patrones en los conceptos de las facturas.
- **Fase 4: Multi-perfil y Colaboración:** Evolución hacia un modelo de "Estudio Creativo" que permita a varios autónomos colaborar en un mismo proyecto, manteniendo balances de gastos compartidos y liquidaciones internas.
- **Fase 5: Conectividad Externa con Gestorías:** Desarrollo de un portal de exportación cifrada para que el asesor fiscal del cliente pueda descargar los

libros de facturas directamente en formatos compatibles con los sistemas de la AEAT.

3.7. Metodología de Trabajo e Interacción

El desarrollo se rige por **Metodologías Ágiles**, permitiendo realizar iteraciones constantes sobre el diseño y la base de datos. Esta transparencia permite a **bruma.studio** validar las interfaces gráficas y la estructura de datos antes de proceder a la fase de programación pesada, minimizando riesgos de desviación de costes y asegurando que el producto final se ajuste milimétricamente a su flujo de trabajo diario.

4. Justificación y Objetivos del Proyecto

4.1. Justificación: La Necesidad de una Soberanía Financiera Real

El ejercicio de la actividad profesional por cuenta propia en el paradigma actual de la economía digital conlleva una carga administrativa que, estadísticamente, llega a eclipsar el tiempo dedicado a la actividad productiva. Tras un análisis exhaustivo del sector de las *Fintech* y las herramientas de planificación de recursos empresariales actuales, se ha identificado una **disfunción estructural crítica**: la fragmentación extrema de la información contable.

AutonoStack se justifica como una respuesta de ingeniería de software a este desorden operativo, basando su relevancia estratégica en los siguientes vectores de valor:

- **Centralización y Verdad Única (Single Source of Truth):** El profesional autónomo suele operar en un estado de dispersión informativa, navegando entre aplicaciones bancarias, hojas de cálculo manuales y repositorios de facturas físicas. La plataforma implementa una arquitectura que permite la convergencia de ingresos y gastos en un **punto único de verdad**, eliminando errores de duplicidad y proporcionando una visión analítica inmediata del rendimiento neto.
- **Análisis Granular por Proyectos:** A diferencia de las herramientas contables convencionales que agrupan los gastos únicamente por criterios cronológicos, AutonoStack introduce una capa de asociación lógica de transacciones a proyectos específicos. Esta funcionalidad es vital para que el profesional identifique con precisión qué clientes o sectores de actividad son realmente rentables tras deducir los costes operativos y el tiempo invertido asociados a cada encargo.
- **Mitigación de Riesgos mediante Previsión Fiscal Preventiva:** La capacidad de automatizar y anticipar el cálculo del IVA trimestral y las retenciones de IRPF elimina la incertidumbre inherente a los cierres contables. Esta planificación financiera proactiva permite al usuario evitar sorpresas impositivas y gestionar su liquidez con una antelación que las herramientas tradicionales no permiten.
- **Democratización de la Inteligencia de Negocio:** La motivación última del proyecto reside en poner al alcance del trabajador individual herramientas de control y análisis de datos profesionales que, hasta la fecha, estaban reservadas para estructuras empresariales con departamentos financieros dedicados.

4.2. Objetivos del Proyecto

Para garantizar que el desarrollo de AutonoStack cumpla con los estándares de robustez, seguridad y escalabilidad exigidos en un entorno de consultoría profesional, se han definido los siguientes niveles de objetivos:

4.2.1. Objetivo General

Diseñar, desarrollar e implementar una aplicación web **Full Stack funcional y de alto rendimiento**, que centralice la gestión financiera del profesional autónomo y transforme el registro operativo diario en un **dashboard de análisis estratégico de rentabilidad** plenamente accesible.

4.2.2. Objetivos Específicos Técnicos

1. **Arquitectura de Datos Robusta:** Implementar un sistema de persistencia mediante una base de datos relacional **PostgreSQL** (vía Supabase), asegurando la integridad referencial de las facturas y optimizando la velocidad de respuesta en consultas de balances complejos.
2. **Seguridad y Blindaje de Privacidad:** Establecer un sistema de autenticación de alta seguridad y, fundamentalmente, aplicar políticas de **Row Level Security (RLS)** para garantizar que la información financiera sea estrictamente privada y accesible de forma exclusiva por su propietario.
3. **Desarrollo de Interfaces Reactivas:** Construir una **Single Page Application (SPA)** utilizando **React** que permita una interacción fluida y dinámica, minimizando los tiempos de latencia durante el registro masivo de facturas y la renderización de gráficos estadísticos.
4. **Entorno de Exploración (Modo Demo):** Desarrollar una capa de datos simulada que permita a potenciales usuarios explorar las capacidades

analíticas de la herramienta mediante *mock data*, sin necesidad de exponer datos reales ni completar procesos de registro iniciales.

4.2.3. Objetivos de Accesibilidad y Usabilidad (Ingeniería de Interfaz)

Como prioridad transversal del proyecto, AutonoStack se marca objetivos que superan la funcionalidad básica para garantizar una experiencia inclusiva:

1. **Cumplimiento Estricto WCAG 2.1 Nivel AA:** Asegurar que todos los componentes del dashboard sean operables mediante teclado, posean etiquetas semánticas correctas para lectores de pantalla y mantengan ratios de contraste lumínico superiores a 4.5:1.
2. **Diseño Universal y Adaptabilidad (Responsiveness):** Garantizar una experiencia de usuario (UX) consistente en dispositivos móviles y de escritorio, permitiendo que el autónomo registre un gasto mediante una captura rápida en su móvil con la misma fiabilidad con la que analiza un informe trimestral en su PC.
3. **Optimización de la Carga Cognitiva:** Diseñar una interfaz limpia y minimalista que priorice la jerarquía visual de los datos críticos (saldo neto e IVA pendiente), evitando el ruido visual y la complejidad innecesaria de los programas de contabilidad tradicionales.

5. Ingeniería de requisitos del aplicativo

La ingeniería de requisitos de **AutonoStack** trasciende la mera enumeración de funcionalidades para constituirse como el plano técnico de una solución de consultoría de alto nivel. Este apartado detalla el exhaustivo proceso de traducción de las necesidades estratégicas de **bruma.studio** en especificaciones técnicas de software. El éxito del aplicativo radica en la simbiosis entre el comportamiento esperado (Requisitos Funcionales) y los atributos de calidad (Requisitos No Funcionales) que garantizan la robustez del sistema en un entorno de producción profesional.

5.1. Requisitos Funcionales (RF)

Los requisitos funcionales definen las capacidades operativas que el aplicativo debe poseer para resolver la problemática de la gestión financiera del autónomo. Se han organizado en módulos lógicos que reflejan la arquitectura interna del sistema.

Módulo 1: Seguridad y Gestión de Identidad

Este módulo constituye el perímetro de defensa de la plataforma, garantizando que el acceso a la información financiera sea estrictamente granular.

- **RF-01. Autenticación Perimetral:** Implementación de un sistema de registro e inicio de sesión seguro mediante **Supabase Auth**. El sistema diferencia dinámicamente tres perfiles: el Administrador (control de plataforma), el Usuario Autónomo (propietario de datos) y el Invitado (acceso demo).
- **RF-07. Panel de Moderación Administrativa:** Conjunto de herramientas exclusivas para el rol administrador que permiten la monitorización técnica y

gestión de altas/bajas de usuarios sin vulnerar el secreto profesional, al carecer de acceso a los registros financieros privados.

- **RF-08. Modo Demo con Persistencia Simulada:** Ruta pública diseñada para la captación de usuarios donde se interactúa con un dashboard precargado con *mock data*, permitiendo explorar la propuesta de valor sin necesidad de un registro previo.

Módulo 2: Núcleo de Gestión Financiera (Core)

Representa el motor contable de AutonoStack, donde se procesa el flujo de caja del estudio creativo.

- **RF-02. Gestión Integral de Facturación (CRUD):** Capacidad de realizar operaciones de creación, lectura, actualización y borrado de facturas emitidas y recibidas. Se capturan campos críticos como fechas de devengo, tipos impositivos e identificación de terceros.
- **RF-03. Categorización Fiscal de Gastos:** Sistema de etiquetado de transacciones según categorías desgravables (suministros, servicios externos, material de oficina) para facilitar la posterior deducción fiscal.
- **RF-04. Estructuración y Rentabilidad por Proyectos:** El sistema permite agrupar transacciones bajo el concepto de "Proyecto", calculando automáticamente el beneficio neto por encargo, una necesidad crítica detectada en la consultoría con **bruma.studio**.

Módulo 3: Inteligencia de Datos y Reportes

Transforma el dato operativo en información estratégica para la toma de decisiones.

- **RF-05. Dashboard Analítico Interactivo:** Interfaz gráfica que muestra en tiempo real la salud financiera del autónomo mediante comparativas de

ingresos frente a gastos y una previsión automatizada de la carga de IVA trimestral.

- **RF-06. Generación de Informes Exportables:** Motor de renderizado de documentos PDF que consolida la información financiera filtrada por periodos o proyectos, lista para su entrega a gestorías o inspecciones.

Módulo 4: Gestión de Activos y Justificantes Digitales

Asegura la validez legal de la contabilidad mediante el almacenamiento documental.

- **RF-09. Almacenamiento en la Nube:** Vinculación directa de cada registro de gasto con su archivo justificante alojado en **Supabase Storage**, garantizando la integridad documental requerida por la normativa vigente.
- **RF-10. Visor de Documentos Integrado:** Implementación de una interfaz de previsualización que permite consultar los archivos PDF o imágenes adjuntas sin necesidad de descargar el archivo o salir de la sesión activa.
- **RF-11. Control de Integridad de Archivos:** Sistema de validación de formatos (MIME types) para prevenir la subida de archivos corruptos o maliciosos que puedan comprometer el servidor.

Módulo 5: Configuración y Cumplimiento Normativo

Permite la adaptación del sistema a la casuística fiscal de cada profesional.

- **RF-12. Gestión de Tipos Impositivos:** Configuración flexible de porcentajes de IVA (4%, 10%, 21%) y retenciones de IRPF, adaptando los cálculos al epígrafe específico del autónomo.
- **RF-13. Perfil Fiscal Personalizado:** Almacenamiento de los datos fiscales del propietario (NIF, domicilio social) para su inclusión automática en la cabecera de los informes y exportaciones legales.

Módulo 1: Seguridad y Gestión de Identidad

ID	Nombre	Descripción	Prioridad	Estado
RF-01	Autenticación Perimetral	Registro e inicio de sesión seguro vía Supabase Auth con roles diferenciados (Admin, Autónomo, Demo).	Alta	Completado
RF-07	Panel de Moderación	Herramientas administrativas para gestión de cuentas y supervisión técnica sin acceso a datos privados.	Media	Completado
RF-08	Modo Demo	Acceso público a una instancia con persistencia simulada (*mock data*) para exploración sin registro.	Baja	Completado

Módulo 2: Núcleo de Gestión Financiera (Core)

ID	Nombre	Descripción	Prioridad	Estado
RF-02	Gestión CRUD Facturación	Mantenimiento completo de facturas emitidas y recibidas incluyendo serie, cliente e impuestos.	Alta	Completado
RF-03	Categorización Fiscal	Clasificación de gastos desgravables por categorías predefinidas con soporte para adjuntos (PDF/JPG).	Alta	Completado
RF-04	Estructuración por Proyectos	Asociación lógica de transacciones a proyectos específicos para el cálculo de márgenes individuales.	Alta	Completado

Módulo 3: Inteligencia de Datos y Reportes

ID	Nombre	Descripción	Prioridad	Estado
RF-05	Dashboard Interactivo	Visualización en tiempo real de flujos de caja y previsión automatizada de IVA trimestral.	Alta	Completado
RF-06	Generación de Informes	Exportación de resúmenes financieros en formato PDF con filtrado por fecha, categoría o proyecto.	Media	Completado

Módulo 4: Gestión de Activos y Justificantes Digitales

ID	Nombre	Descripción	Prioridad	Estado
RF-09	Almacenamiento en la Nube	Vinculación de cada registro de gasto con un archivo físico alojado en **Supabase Storage** para garantizar su validez ante auditorías.	Alta	Completado

RF-10	Visor de Documentos	Implementación de un previsualizador integrado que permita consultar facturas en PDF o imagen sin abandonar el ecosistema de la aplicación.	Media	Completado
RF-11	Control de Integridad de Archivos	Sistema de validación de formatos (MIME types) para asegurar que solo se suban documentos legibles y libres de scripts maliciosos.	Alta	Completado

Módulo 5: Configuración y Cumplimiento Normativo

ID	Nombre	Descripción	Prioridad	Estado
RF-12	Gestión de Tipos Impositivos	Posibilidad de configurar diferentes tipos de IVA (4%, 10%, 21%) y retenciones de IRPF según la	Alta	Completado

		naturaleza de la actividad del autónomo.		
RF-13	Perfil Fiscal Personalizado	Configuración de datos de facturación propios (NIF, domicilio, razón social) para su inclusión automática en la generación de informes PDF.	Media	Completado

5.2. Requisitos No Funcionales (RNF)

Los requisitos no funcionales definen los estándares de calidad, seguridad y eficiencia que garantizan que la plataforma sea apta para el uso profesional y profesionalmente confiable.

Módulo 6: Calidad de Software e Infraestructura

Este bloque asegura que la base técnica sea sólida, escalable y segura.

- **RNF-01. Seguridad de Datos y Segregación:** Uso de políticas **RLS (Row Level Security)** en PostgreSQL para garantizar el aislamiento físico de los datos entre usuarios, complementado con validación de sesiones mediante **JWT**.

- **RNF-02. Rendimiento y Eficiencia Visual:** El aplicativo debe responder a las interacciones críticas y cargar gráficas analíticas en un tiempo inferior a los dos segundos bajo conexiones estándar.
- **RNF-03. Accesibilidad Universal (Nivel AA):** Compromiso social de cumplir con las pautas **WCAG 2.1**, asegurando contrastes superiores a 4.5:1 y una navegación íntegra mediante teclado para usuarios con diversidad funcional.
- **RNF-04. Interfaz Responsive y Adaptabilidad:** Aplicación de **Tailwind CSS** para asegurar que la experiencia de usuario sea idéntica y eficiente tanto en smartphones (para el registro rápido de tickets) como en ordenadores de sobremesa.
- **RNF-05. Integridad y Consistencia Transaccional:** Implementación de restricciones en base de datos para prevenir duplicidades de facturas y errores de redondeo en los cálculos impositivos.
- **RNF-06. Arquitectura Modular y Escalabilidad:** El frontend se desarrolla mediante componentes de **React** reutilizables, permitiendo una expansión futura del sistema (como la conciliación bancaria) sin necesidad de reescribir el núcleo.

Módulo 7: Disponibilidad y Soporte Técnico

Establece los criterios de resiliencia del servicio ante incidencias.

- **RNF-07. Alta Disponibilidad:** Garantía de un tiempo de actividad del 99.9% mediante la infraestructura distribuida de **Vercel** y **Supabase**, minimizando las ventanas de mantenimiento.
- **RNF-08. Recuperación de Datos (Disaster Recovery):** Implementación de copias de seguridad automáticas diarias gestionadas por el motor de PostgreSQL para asegurar la integridad de la información ante fallos críticos.

- **RNF-09. Documentación Técnica Integrada:** Disponibilidad de manuales de explotación y guías de usuario integradas en la plataforma para facilitar la autonomía del profesional en el uso del software.

Módulo 6: Calidad de Software e Infraestructura

ID	Nombre	Descripción	Prioridad	Estado
RNF-01	Segregación de Datos	Implementación de Row Level Security (RLS) en PostgreSQL y validación de sesiones JWT.	Crítica	Completado
RNF-02	Eficiencia Visual	Carga de componentes y gráficas en menos de 2 segundos bajo condiciones de conexión estándar.	Alta	Completado
RNF-03	Accesibilidad (AA)	Cumplimiento estricto de WCAG 2.1: contraste > 4.5:1 y navegación íntegra por teclado.	Crítica	Completado

RNF-04	Adaptabilidad	Interfaz 100% responsive desarrollada con Tailwind CSS para móviles, tablets y escritorio.	Alta	Completado
RNF-05	Consistencia de Datos	Restricciones de integridad en base de datos para evitar duplicidades y errores de redondeo fiscal.	Alta	Completado
RNF-06	Escalabilidad Front-end	Arquitectura modular basada en componentes React reutilizables para futuras integraciones.	Media	Completado

Módulo 7: Disponibilidad y Soporte Técnico

ID	Nombre	Descripción	Prioridad	Estado
RNF-07	Alta Disponibilidad	Garantizar un tiempo de actividad (Uptime) del 99.9% mediante la infraestructura	Alta	Completado

		distribuida de Vercel y Supabase.		
RNF-08	Recuperación de Datos	Implementación de copias de seguridad automáticas diarias gestionadas por el motor de PostgreSQL para prevenir pérdida de información crítica.	Crítica	Completado
RNF-09	Documentación Técnica Integrada	Disponibilidad de manuales de usuario y técnicos accesibles directamente desde el panel de ayuda para reducir la curva de aprendizaje.	Baja	Completado

6. Roles, Permisos y Matriz de Acceso

La arquitectura de seguridad de **AutonoStack** se fundamenta en un modelo de **Control de Acceso Basado en Roles (RBAC)**, diseñado no solo para restringir el flujo de información, sino para garantizar la integridad financiera y la privacidad absoluta de cada profesional independiente. Este sistema adapta dinámicamente la lógica del servidor y la interfaz de usuario (UI) según el perfil autenticado, cumpliendo con el principio de mínimo privilegio y los estándares de seguridad exigidos en aplicaciones de gestión de datos críticos.

6.1. Diagrama de Casos de Uso y Escenarios Operativos

Para representar visualmente la interacción entre los actores identificados y el ecosistema funcional, se ha elaborado un diagrama de casos de uso que delimita con precisión las fronteras operativas del aplicativo. Este esquema permite identificar cómo los diferentes perfiles activan los procesos de negocio, desde la ingesta de datos brutos (facturas) hasta la generación de inteligencia financiera (dashboard).

El diagrama destaca la **independencia soberana** del Usuario Autónomo en la gestión de su contabilidad y cómo el Administrador opera exclusivamente en una capa de gestión de servicios, mientras que el Invitado queda restringido a una "sandbox" o burbuja de visualización de datos simulados para preservar la integridad del sistema productivo.

6.2. Definición y Caracterización de los Actores del Sistema

Para cubrir tanto las necesidades operativas del profesional como las de mantenimiento técnico y prospección comercial, se han definido tres perfiles con capacidades y responsabilidades estancas:

6.2.1. Actor: Usuario Autónomo (Perfil Operativo Principal)

Representa el núcleo operativo y el destinatario final de la plataforma. Es el profesional que utiliza la herramienta para digitalizar y profesionalizar su gestión económica bajo un modelo de soberanía total sobre su información.

- **Perfil:** Freelancers, consultores o microempresarios que requieren una solución ágil para centralizar su información fiscal y analizar la rentabilidad real de sus esfuerzos por proyecto.
- **Gestión Documental:** Es el único responsable de la introducción y veracidad de sus facturas y justificantes, actuando como el administrador de su propio repositorio documental.
- **Análisis Estratégico:** Utiliza el Dashboard para interpretar su salud financiera, monitorizar el IVA trimestral y tomar decisiones sobre la viabilidad de sus proyectos actuales y futuros.
- **Blindaje de Datos:** Su acceso está protegido por políticas de base de datos que impiden que cualquier otro actor, incluido el equipo técnico, visualice sus registros contables privados.

6.2.2. Actor: Administrador (Gestión Técnica del Ecosistema)

Este perfil no interviene en la contabilidad ni en la operativa financiera del cliente; su misión es velar por la disponibilidad, salud y seguridad del servicio global.

- **Responsabilidades:** Gestión del ciclo de vida de los usuarios (altas, bajas y resolución de incidencias de acceso) y supervisión de la infraestructura técnica.
- **Privacidad por Diseño (Privacy by Design):** Por arquitectura, el administrador tiene restringido el acceso a la capa de datos financieros mediante la configuración de políticas a nivel de fila (RLS) que no incluyen su rol como autorizado para lectura de transacciones.
- **Auditoría:** Supervisión de los logs de acceso para garantizar que el sistema cumpla con las normativas de seguridad y evitar accesos no autorizados.

6.2.3. Actor: Invitado / Modo Demo (Perfil de Exploración)

Representa al usuario potencial que desea evaluar la propuesta de valor de **AutonoStack** sin realizar un proceso de registro formal ni comprometer datos reales.

- **Flujo de Interacción:** Acceso restringido a una instancia de "solo lectura" donde interactúa con datos ficticios (*mock data*).
- **Limitaciones:** No tiene capacidad de persistencia (no puede guardar cambios) ni acceso a funciones de exportación real, lo que permite conocer la potencia analítica de la herramienta de forma segura y efímera.

6.3. Matriz de Responsabilidades y Acceso Técnico

Para asegurar una implementación técnica coherente en el código y en las políticas de la base de datos, se establece la siguiente matriz de permisos cruzada:

MÓDULO / FUNCIONALIDAD	USUARIO AUTÓNOMO	ADMINISTRADOR	INVITADO (DEMO)
Autenticación (Login/Logout)	SÍ	SÍ	NO
Registro de Gastos e Ingresos	SÍ (PROPIOS)	NO (BLOQUEADO)	NO (BLOQUEADO)
Gestión de Proyectos	SÍ (PROPIOS)	NO (BLOQUEADO)	NO (BLOQUEADO)
Visualización Dashboard Real	SÍ	NO (BLOQUEADO)	NO (BLOQUEADO)
Visualización Dashboard Demo	NO	NO	SÍ (SOLO LECTURA)
Subida de Justificantes (Storage)	SÍ	NO	NO

Generación de Informes PDF	SÍ	NO	NO
Gestión de Cuentas (Alta/Baja)	NO	SÍ	NO
Auditoría de Logs Técnicos	NO	SÍ	NO
Acceso a Datos de Terceros (RLS)	**BLOQUEADO**	**BLOQUEADO**	**BLOQUEADO**

6.4. Justificación Técnica y Seguridad de Acceso

La implementación de esta matriz no es meramente organizativa, sino que responde a imperativos legales y de arquitectura de software:

1. **Aislamiento por Diseño:** Se garantiza que el Administrador, a pesar de tener control sobre la infraestructura, carezca de permisos técnicos para visualizar la facturación o los proyectos, cumpliendo con la confidencialidad pactada en el contrato de servicios.
2. **Seguridad RLS (Row Level Security):** Se han configurado reglas en el motor PostgreSQL para que el Usuario Autónomo solo pueda operar sobre las filas que contengan su user_id, garantizando privacidad absoluta incluso ante vulnerabilidades de inyección en el frontend.

3. **Resiliencia en el Modo Demo:** El acceso controlado a datos ficticios permite evaluar la herramienta sin comprometer la base de datos de producción ni generar costes de almacenamiento innecesarios.
4. **Trazabilidad y Conservación:** Todas las interacciones críticas quedan registradas en logs persistentes, asegurando que los registros sean auditables durante el periodo legal de 4 años exigido para normativas fiscales externas.

6.5. Implementación y Garantía de Accesibilidad en la Gestión de Roles

La gestión de roles en **AutonoStack** impacta directamente en la experiencia de usuario (UX) y la accesibilidad:

- **Visibilidad Adaptativa:** La interfaz se simplifica según el rol activo. Un usuario en "Modo Demo" visualizará avisos claros sobre la naturaleza ficticia de los datos, reduciendo la carga cognitiva y evitando confusiones con su contabilidad real.
- **Seguridad Invisible:** Mediante el uso de políticas RLS, el sistema protege al usuario sin añadir barreras de uso complejas. El autónomo nunca podrá acceder a datos ajenos ni siquiera mediante la manipulación manual de URLs.
- **Inclusión Administrativa:** El panel de administración se ha diseñado siguiendo los mismos criterios de alto contraste y navegación por teclado que el dashboard principal, asegurando que la gestión técnica sea inclusiva para cualquier profesional.

7. Diseño de la Arquitectura de Datos y Lógica de Negocio

Esta sección técnica constituye el núcleo de la ingeniería de software del proyecto. Tras haber definido el marco estratégico y los requisitos, procedemos al diseño detallado de la infraestructura de datos y la arquitectura lógica que sustenta a **AutonoStack**. La robustez del aplicativo no reside solo en su interfaz, sino en una base de datos diseñada para la **resiliencia, la integridad atómica y el blindaje absoluto de la información financiera**.

La arquitectura de datos ha sido diseñada bajo el paradigma de **modelado relacional estricto**, priorizando la integridad referencial y la escalabilidad de las consultas financieras complejas. El uso de **PostgreSQL** a través de la infraestructura de **Supabase** permite delegar la persistencia en un motor industrial, mientras se implementa una lógica de seguridad profunda mediante políticas de acceso que garantizan la soberanía del dato.

7.1. Modelo Entidad-Relación (E-R)

El diseño del esquema de la base de datos se articula en torno a la entidad de usuario, vinculando de forma jerárquica sus proyectos y transacciones. Este modelo asegura que no existan huérfanos de datos y que la trazabilidad sea absoluta desde el origen de la factura hasta su reflejo en el balance impositivo.

La relación se define como **1:N (Uno a Varios)**: un perfil de autónomo puede poseer múltiples proyectos, y cada proyecto puede contener una infinidad de transacciones financieras, manteniendo siempre la clave externa (user_id) como el ancla de seguridad del sistema.

7.1.1. Diccionario de Datos Detallado

A continuación, se describen las especificaciones técnicas de las tablas que componen el ecosistema de **AutonoStack**:

Tabla 7.1: Especificaciones de la Tabla profiles

Columna	Tipo de Dato	Restricción	Descripción Técnica
id	uuid	PRIMARY KEY	Referencia única vinculada al esquema auth.users de Supabase.
email	text	UNIQUE / NOT NULL	Identificador de comunicación y acceso del autónomo.
full_name	text	NOT NULL	Nombre completo o razón social para validez fiscal.
role	text	CHECK (role IN (...))	Define las capacidades operativas (Admin, Autónomo, Demo).
avatar_url	text	NULLABLE	Puntero al objeto de imagen en el Bucket de Storage.
updated_at	timestamp	DEFAULT now()	Registro cronológico para auditoría de cambios.

Tabla 7.2: Especificaciones de la Tabla projects

Columna	Tipo de Dato	Restricción	Descripción Técnica
id	uuid	PRIMARY KEY	Identificador único universal del proyecto creativo.
user_id	uuid	FOREIGN KEY	Vinculación obligatoria con profiles.id (Estructura RLS).
name	text	NOT NULL	Nombre comercial del proyecto o cliente asignado.
description	text	NULLABLE	Metadatos descriptivos sobre el alcance del trabajo.
status	text	DEFAULT 'active'	Estado vital del proyecto (active, completed, archived).
created_at	timestamp	DEFAULT now()	Fecha de apertura contable del proyecto.

Tabla 7.3: Especificaciones de la Tabla transactions

Columna	Tipo de Dato	Restricción	Descripción Técnica
id	uuid	PRIMARY KEY	Identificador único de la operación financiera.
user_id	uuid	FOREIGN KEY	Garantiza que la transacción pertenece a un usuario único.
project_id	uuid	FOREIGN KEY	Asociación obligatoria para el cálculo de márgenes netos.
type	text	CHECK (type IN (...))	Clasificación binaria: 'ingreso' (factura) o 'gasto' (ticket).
amount	numeric(12,2)	NOT NULL	Importe base de la operación (precisión decimal fiscal).
tax_rate	numeric(4,2)	DEFAULT 21.00	Porcentaje de IVA aplicable según normativa.
category	text	NOT NULL	Clasificación para desgravación (Suministros, Servicios, etc.).

date	date	NOT NULL	Fecha de devengo del impuesto o gasto.
file_url	text	NULLABLE	Vínculo al justificante físico en la nube (PDF/JPG).

7.1.2. Matriz de Trazabilidad Requisitos-Datos

Para garantizar que el diseño de la base de datos no es arbitrario, sino que responde a las necesidades estratégicas de **bruma.studio**, se presenta la matriz que vincula los requisitos funcionales con las entidades físicas del sistema.

ID Requisito	**Descripción del Requisito**	**Entidad / Tabla Relacionada**	**Justificación de Almacenamiento**
RF-01	Autenticación y Perfiles	profiles	Persistencia de metadatos de usuario tras el handshake de Auth.
RF-02	Gestión CRUD Facturación	transactions	Registro atómico de cada movimiento financiero con precisión decimal.
RF-04	Estructuración por Proyectos	projects	Agrupación lógica mediante claves foráneas para análisis de margen.

RF-09	Almacenamiento en la Nube	transactions (file_url)	Puntero de referencia al objeto binario en el Bucket de Storage.
RF-12	Gestión de Tipos Impositivos	transactions (tax_rate)	Almacenamiento del multiplicador fiscal para cálculos de cuota.

7.2. Scripts de Creación de Estructura (DDL)

Para garantizar la replicabilidad del entorno y la consistencia de la base de datos en producción, se presentan los scripts de definición de datos (**Data Definition Language**). Se han incluido índices estratégicos para optimizar la velocidad de respuesta en consultas de agregación financiera.

```
-- 1. Creación de la tabla de Perfiles (Extensión de Auth)

CREATE TABLE profiles (

  id UUID REFERENCES auth.users ON DELETE CASCADE PRIMARY KEY,

  email TEXT UNIQUE NOT NULL,

  full_name TEXT NOT NULL,

  role TEXT DEFAULT 'autonomo' CHECK (role IN ('autonomo', 'admin', 'demo')),

  avatar_url TEXT,

  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()

);
```

```
-- 2. Creación de la tabla de Proyectos
CREATE TABLE projects (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  user_id UUID REFERENCES profiles(id) ON DELETE CASCADE NOT NULL,
  name TEXT NOT NULL,
  description TEXT,
  status TEXT DEFAULT 'active' CHECK (status IN ('active', 'completed', 'archived')),
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
-- 3. Creación de la tabla de Transacciones (Núcleo Contable)
CREATE TABLE transactions (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  user_id UUID REFERENCES profiles(id) ON DELETE CASCADE NOT NULL,
  project_id UUID REFERENCES projects(id) ON DELETE CASCADE NOT NULL,
  type TEXT NOT NULL CHECK (type IN ('ingreso', 'gasto')),
  amount NUMERIC(12,2) NOT NULL,
  tax_rate NUMERIC(4,2) DEFAULT 21.00,
  category TEXT NOT NULL,
  date DATE NOT NULL,
  file_url TEXT,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW());
```

```
-- 4. Índices de Optimización (RNF-02)

CREATE INDEX idx_transactions_user_id ON transactions(user_id);

CREATE INDEX idx_transactions_project_id ON transactions(project_id);

CREATE INDEX idx_projects_user_id ON projects(user_id);
```

7.2.1. Diccionario de Tipos Enumerados (Enums)

Para evitar la inconsistencia de datos y asegurar la integridad de los estados del sistema, se han definido tipos de datos personalizados que restringen los valores posibles en columnas críticas.

Tipo Enumerado	**Valores Permitidos**	**Propósito en el Negocio**
user_role	autonomo, admin, demo	Segregación de privilegios y lógica de interfaz.
project_status	active, completed, archived	Gestión del ciclo de vida de los encargos de bruma.studio.
transaction_type	ingreso, gasto	Clasificación binaria para el motor de balance neto.

```
-- Definición de Tipos Personalizados

CREATE TYPE user_role AS ENUM ('autonomo', 'admin', 'demo');

CREATE TYPE project_status AS ENUM ('active', 'completed', 'archived');

CREATE TYPE transaction_type AS ENUM ('ingreso', 'gasto');
```

```
-- Aplicación en la estructura (Actualización de DDL)

ALTER TABLE profiles ALTER COLUMN role TYPE user_role;

ALTER TABLE projects ALTER COLUMN status TYPE project_status;

ALTER TABLE transactions ALTER COLUMN type TYPE transaction_type;
```

7.3. Implementación de Seguridad a Nivel de Fila (Row Level Security)

7.3.1. Concepto y Necesidad de RLS: El Modelo de Confianza Cero

En las arquitecturas de software convencionales, la seguridad suele delegarse exclusivamente en una capa intermedia de servidor (Middleware). Sin embargo, para una plataforma de alta criticidad financiera como **AutonoStack**, se ha implementado un modelo de **Seguridad de Confianza Cero** utilizando **RLS nativo de PostgreSQL** a través de Supabase.

El RLS permite que la base de datos sea "consciente" de la identidad del actor que realiza la consulta en cada milisegundo. Sin esta configuración, un usuario malintencionado podría, mediante la manipulación de la API o inyecciones de código, acceder a la tabla de transacciones de otro profesional. Con RLS, la lógica de seguridad reside en el núcleo del motor de datos: la base de datos deniega el acceso de forma atómica si el registro no pertenece al propietario de la sesión.

7.3.2. Configuración de la Política de Aislamiento

Para asegurar que los datos de **bruma.studio** permanezcan estancos y protegidos de cualquier otro usuario de la plataforma, se han ejecutado los siguientes comandos de blindaje técnico:

1. **Habilitación de Seguridad:** ALTER TABLE transactions ENABLE ROW LEVEL SECURITY;
2. **Definición de la Política de Identidad:** Se utiliza la función auth.uid(), la cual extrae de forma criptográficamente segura el ID del usuario desde el token **JWT (JSON Web Token)** firmado que envía el navegador tras el login.

Lógica de la Política de Privacidad (SQL)

```
-- Política para permitir a los autónomos acceder exclusivamente a sus propios datos financieros

CREATE POLICY "Privacidad total: acceso solo al propietario"

ON transactions

FOR ALL -- Aplica a SELECT, INSERT, UPDATE y DELETE

USING (auth.uid() = user_id);
```

7.3.3. Funcionamiento del Filtro Automático Invisible

La potencia de esta implementación reside en su transparencia para el desarrollador y el usuario. Cuando un autónomo con el ID A-001 solicita visualizar sus transacciones, Supabase intercepta la consulta y añade automáticamente una cláusula **WHERE invisible**: WHERE user_id = 'A-001'.

- **Integridad Garantizada:** Si el usuario intenta forzar manualmente una consulta sobre el ID de un tercero, el motor de PostgreSQL devolverá un conjunto de resultados vacío, protegiendo la confidencialidad pactada en el marco contractual.

7.3.4. Políticas de Seguridad de Objetos (Bucket RLS)

Dada la naturaleza sensible de las facturas y tickets en PDF/JPG, no basta con proteger la base de datos; el almacenamiento físico de archivos (Storage) debe estar blindado bajo el mismo modelo de **Confianza Cero**.

Comando de Seguridad	**Acción Protegida**	**Lógica de la Política**
Enable RLS (Bucket)	Acceso a archivos	El Bucket "justificantes" solo responde a usuarios autenticados.
Policy: Owner Access	SELECT / INSERT	(storage.foldername(name))[1] = auth.uid()::text

Justificación Técnica: Esta configuración garantiza que, incluso si alguien descubre la URL directa de una factura de **bruma.studio**, el servidor de almacenamiento denegará la descarga si el token del solicitante no coincide con el ID de la carpeta del propietario.

7.4. Capa de Lógica de Negocio: Algoritmos de Inteligencia Financiera

Para transformar los datos operativos en indicadores estratégicos, el sistema implementa una capa de cálculo dinámico.

Tabla 7.4: Algoritmos de Cálculo y Propósito

Indicador	Algoritmo Lógico (Descripción)	Propósito Funcional
Balance Neto	Suma total de los ingresos menos la suma total de los gastos registrados.	Determina la solvencia real y la liquidez disponible del estudio creativo en tiempo real.
IVA Repercutido	Suma de los importes de cada ingreso multiplicados por su tipo de IVA correspondiente.	Previsión automatizada de la deuda tributaria trimestral para evitar sorpresas fiscales.
IVA Soportado	Suma de los importes de cada gasto multiplicados por su tipo de IVA correspondiente.	Cálculo preciso de la cuota deducible acumulada basada en gastos desgravables.
Margen Neto	Diferencia entre ingresos y gastos del proyecto, dividida entre los ingresos del proyecto y multiplicada por cien.	Identificación estratégica de la rentabilidad individual por cada proyecto o unidad de negocio.

7.5. Estrategia de Persistencia y Caché de Alto Rendimiento

Para cumplir con los requisitos de **Eficiencia Visual** (carga < 2s), se ha diseñado una estrategia de gestión de estado híbrida:

1. **Sincronización con React Query:** Permite que el dashboard muestre datos almacenados en caché instantáneamente mientras valida los cambios con Supabase en segundo plano.
2. **Optimistic Updates:** Al registrar un gasto, la interfaz se actualiza de forma inmediata, proyectando el resultado en las gráficas antes de recibir la confirmación del servidor, lo que elimina la percepción de latencia en movilidad.
3. **Gestión de Binarios:** Implementación de compresión de imágenes *on-the-fly* antes de la subida a Storage, optimizando el espacio de almacenamiento y la velocidad de carga de los justificantes.

7.6. Lógica de Agregación: Vistas SQL para el Dashboard

Para cumplir con el requisito de carga acelerada (**RNF-02**), se han implementado **Vistas de Base de Datos**. Esto permite que los cálculos complejos (como el IVA trimestral) se realicen en el motor de PostgreSQL y no en el navegador del usuario, optimizando el rendimiento.

Tabla 7.6: Vistas de Inteligencia de Negocio

Nombre de la Vista	Función de Negocio	Definición Lógica (SQL)
vw_resumen_trimestral	Previsión de IVA 303	Agrupa ingresos/gastos por EXTRACT(QUARTER FROM date).
vw_rentabilidad_proyectos	Ranking de beneficios	Calcula SUM(amount) agrupado por project_id.

```
-- Vista para el Dashboard: Cálculo de IVA por Trimestre

CREATE VIEW vw_resumen_trimestral AS

SELECT

 user_id,

 EXTRACT(QUARTER FROM date) AS trimestre,

 SUM(CASE WHEN type = 'ingreso' THEN amount * (tax_rate/100) ELSE 0 END)
AS iva_repercutido,

SUM(CASE WHEN type = 'gasto' THEN amount * (tax_rate/100) ELSE 0 END) AS
iva_soportado

FROM transactions

GROUP BY user_id, trimestre;
```

7.7. Integridad Referencial y Acciones de Disparo (Triggers)

Para asegurar que el sistema sea "autolimpiable" y consistente, se han definido acciones de integridad que gestionan el borrado de datos en cascada.

Relación	Acción en DELETE	Justificación
profiles ➔ projects	CASCADE	Si un usuario se da de baja, sus proyectos se eliminan por privacidad.
projects ➔ transactions	RESTRICT	Impide borrar un proyecto si tiene facturas vinculadas (evita orfandad fiscal).
transactions ➔ storage	MANUAL/HOOK	Se activa una función para eliminar el archivo físico al borrar el registro.

8. Diseño de la Experiencia de Usuario (UX) e Interfaz (UI)

La arquitectura de **AutonoStack** trasciende la mera búsqueda de la excelencia estética para consolidarse como una **obra de ingeniería de interacción de alto nivel**. Esta dimensión del proyecto se ha gestado bajo el imperativo categórico de

mitigar la fatiga administrativa crónica que asola al profesional independiente, transmutando la aridez de la gestión contable en un ecosistema fluido, intuitivo y profundamente empoderador.

Siguiendo una metodología de vanguardia centrada en el factor humano, se ha priorizado la eficiencia operativa y la soberanía visual de los datos, asegurando que cada píxel en pantalla actúe como una extensión estratégica del flujo de trabajo de **bruma.studio**.

8.1. Metodología y Estrategia de Diseño UX: La Neuroergonomía del Control

La arquitectura UX de **AutonoStack** se fundamenta en la teoría de la **reducción absoluta de la carga cognitiva**. En un escenario donde el profesional debe procesar variables numéricas de alta complejidad —como el IVA repercutido, retenciones de IRPF y márgenes de contribución—, la interfaz se manifiesta como un filtro inteligente que jerarquiza la información con precisión quirúrgica.

Se ha aplicado un enfoque de **Diseño Universal y Neuroergonomía**, garantizando que la plataforma resulte una herramienta de uso natural tanto para perfiles técnicos como para los directivos creativos de **bruma.studio**, quienes demandan una gestión simplificada pero sin fisuras analíticas. El sistema se basa en la metodología de **Atomic Design**, donde cada átomo (botones, inputs) escala hacia organismos complejos (formularios, widgets) manteniendo una coherencia semántica total.

Tabla 8.1: Criterios de Excelencia en el Diseño UX

Criterio UX	Implementación Técnica y Conceptual	Beneficio Estratégico para el Autónomo
Soberanía Visual	Jerarquía tipográfica avanzada con la fuente **Inter**, optimizando el reconocimiento de cifras críticas.	Interpretación instantánea del estado de liquidez y solvencia sin error interpretativo.
Anticipación Fiscal	Algoritmos predictivos visualizados en widgets de cálculo preventivo de IVA.	Erradica la incertidumbre financiera antes de los cierres, permitiendo planificación proactiva.
Ecosistema Coherente	Biblioteca de componentes atómicos de **React**, garantizando una gramática visual uniforme.	Reduce la curva de aprendizaje; dominio de la herramienta en una sola sesión operativa.
Validación Dinámica	Microinteracciones de alto rendimiento para validaciones fiscales en tiempo real.	Escudo contra la inconsistencia de datos y errores humanos en el registro tributario.

8.2. El Corazón Analítico: El Dashboard como Centro de Inteligencia Visual

El pilar fundamental y la mayor ventaja competitiva de **AutonoStack** reside en su **Dashboard Analítico**, una pieza de ingeniería visual diseñada para convertir el dato bruto en una ventaja competitiva de alto valor. Mientras que las soluciones de gestión tradicionales se limitan a ser meros repositorios de documentos, esta plataforma se posiciona como un sistema de **Inteligencia de Negocio (BI)** que prioriza la interpretación gráfica para una toma de decisiones informada y veloz.

8.2.1. Arquitectura del Dashboard de Alto Rendimiento

El centro de control integra componentes de visualización de vanguardia (**Tremor / Recharts**) que transmutan la complejidad de la base de datos en una narrativa visual accionable:

- **KPI Strategic Cards:** Paneles de control situados en la zona de máximo impacto visual que presentan el saldo neto, los ingresos brutos y los gastos operativos con una claridad meridiana.
- **Cashflow Dynamic Chart:** Una gráfica de áreas de alta resolución que mapea la evolución temporal de la tesorería, permitiendo detectar patrones de estacionalidad y picos de rentabilidad.
- **Preventive Tax Widget:** Un componente crítico que monitoriza el IVA acumulado en tiempo real, actuando como un faro de seguridad financiera que evita sorpresas impositivas al final del ejercicio.

8.3. Iconografía: Semántica Visual y Claridad Simbólica

La iconografía ha sido seleccionada bajo criterios de **funcionalidad técnica y minimalismo suizo**. Se utiliza la librería **Lucide-React**, caracterizada por trazos consistentes de $2px$ y una geometría limpia que complementa perfectamente la tipografía **Inter**.

- **Propósito Semántico:** Cada icono actúa como un ancla cognitiva. No son elementos decorativos, sino señales que guían el ojo del usuario hacia acciones específicas sin necesidad de leer etiquetas de texto, optimizando la velocidad de procesamiento.
- **Estilo Visual:** Se ha optado por un estilo *outline* (contorno) para mantener la ligereza visual, evitando que la interfaz se sienta "pesada", incluso en pantallas con alta densidad de datos.

Tabla 8.2: Familia de Iconos y Justificación

Familia de Iconos	Representación (Ejemplos)	Justificación Semiótica
Financieros	TrendingUp, Receipt, Wallet	Facilitan la identificación de transacciones y estados de cuenta de forma inmediata.
Navegación	LayoutDashboard, Briefcase, Files	Separan claramente las áreas de análisis, gestión de proyectos y archivo documental.

Acción	PlusCircle, Download, Search	Indican interacciones proactivas con un alto contraste visual para atraer la atención.

8.4. Desglose de Pantallas Principales: Ingeniería de Vistas

La experiencia de **AutonoStack** se articula a través de nodos operativos diseñados para cubrir el ciclo de vida completo de la gestión del autónomo, asegurando que la transición entre el análisis macro y la edición micro sea imperceptible.

8.4.1. Dashboard (Vista General Estratégica)

Es la pantalla de aterrizaje y el cerebro del sistema. Su diseño prioriza la ley de la proximidad y la jerarquía de la información. Los widgets de mayor relevancia (Saldo e IVA) ocupan el cuadrante superior izquierdo, siguiendo el patrón de lectura en "F" para garantizar que lo más urgente sea lo primero en procesarse.

8.4.2. Libro de Transacciones (Gestión Operativa)

Diseñada bajo el concepto de **tabla de alta densidad**. Permite la visualización de cientos de registros sin pérdida de rendimiento. Incluye filtros rápidos por fecha, categoría y proyecto, además de un buscador global indexado que reduce el tiempo de localización de facturas en un 70%.

8.4.3. Centro de Proyectos (Análisis de Rentabilidad)

Esta pantalla transforma la contabilidad en estrategia. Cada proyecto se visualiza como una entidad independiente con su propio balance de ingresos y gastos,

permitiendo al creativo evaluar qué trabajos son realmente lucrativos mediante el cálculo del margen:

El margen se calcula restando los gastos a los ingresos, dividiendo el resultado obtenido entre los ingresos totales y multiplicando la cifra resultante por cien para obtener el porcentaje de rentabilidad.

8.4.4. Bóveda Documental (Seguridad y Trazabilidad)

Un entorno dedicado a la gestión de archivos adjuntos. Aquí, el usuario puede visualizar de forma rápida todos los justificantes alojados en **Supabase Storage**, garantizando que cada apunte contable tenga su respaldo legal accesible sin fricciones.

8.5. Diseño de Interfaz (UI) y Semiótica del Color

La estética de **AutonoStack**, implementada mediante la potencia de **Tailwind CSS**, destila una identidad visual de **"Arquitectura de Control"**: limpia, robusta y libre de distracciones innecesarias.

Tabla 8.3: Semiótica Cromática Aplicada a la Gestión Financiera

Color Dominante	Contexto de Aplicación	Justificación Técnica de Usabilidad
Verde Digital (#4ADE80)	Ingresos y Saldos Positivos	Asociación psicológica universal con salud financiera y crecimiento de capital.

Azul Profundo (#1D4ED8)	Navegación y Estructura	Transmite rigor técnico institucional, ideal para la concentración administrativa densa.
Naranja Ambarino (#F59E0B)	Alertas de IVA e Impuestos	Identificador de atención proactiva; señala hitos sin generar estrés o pánico visual.
Gris Antracita	Tipografía y Datos	Maximiza el contraste lumínico (ratio $> 4.5:1$), garantizando legibilidad absoluta.

8.6. Ingeniería de Accesibilidad (Compromiso WCAG 2.1 AA)

En consonancia con los valores de inclusión digital, el diseño de la interfaz ha sido auditado para cumplir con los estándares internacionales más exigentes:

- **Navegación Semántica y ARIA:** Empleo de etiquetas **HTML5** y descriptores **ARIA** para garantizar que los lectores de pantalla traduzcan el pulso financiero a usuarios con diversidad funcional.
- **Independencia del Color:** Los indicadores no confían solo en el color; cada métrica se apoya en iconografía semántica para asegurar la comprensión en casos de daltonismo.
- **Operabilidad por Teclado:** Implementación de estados de foco visual de alto contraste para permitir una gestión profesional sin dependencia del ratón.

8.7. Paradigma Adaptativo: Universalidad Mobile-First

Dada la naturaleza nómada del autónomo moderno, **AutonoStack** ha sido diseñado bajo el paradigma de la **Universalidad Adaptativa**. La interfaz no se limita a "encogerse", sino que se reconfigura dinámicamente.

Tabla 8.4: Comportamiento Responsive de los Componentes

Componente UI	Adaptación en Escritorio (Desktop)	Adaptación en Movilidad (Mobile)
Navegación	Panel lateral persistente con etiquetas.	Barra inferior tipo "Tab Bar" optimizada para pulgares.
Visualización	Tablas multicolumna con filtros complejos.	Tarjetas informativas expandibles mediante gestos táctiles.
Entrada de Datos	Rejilla en paralelo para agilizar entrada masiva.	Formularios verticales de una columna para evitar fatiga visual.
Documentación	Visor integrado en ventana lateral.	Apertura en modal de pantalla completa con controles zoom.

8.8. Tabla Resumen de Pantallas Principales del Ecosistema

Pantalla	Propósito Estratégico	Elemento Diferenciador	Impacto en el Usuario
Dashboard	Inteligencia de Negocio y Visión 360°	Gráficas dinámicas de flujo de caja en tiempo real.	Toma de decisiones basada en datos, no en intuiciones.
Transacciones	Eficiencia en el Registro Operativo	Sistema de filtrado multivariable y búsqueda instantánea.	Reducción drástica del tiempo dedicado a la "burocracia".
Proyectos	Control de Rentabilidad Granular	Cálculo de margen neto automatizado por unidad de negocio.	Identificación inmediata de los clientes más rentables.
Documentos	Seguridad Jurídica y Auditoría	Visor de archivos en la nube integrado sin descarga.	Paz mental ante posibles inspecciones o revisiones fiscales.
Configuración	Personalización del Perfil Fiscal	Ajuste dinámico de tipos impositivos (IVA/IRPF).	Adaptabilidad total a la situación legal del autónomo.

9. Especificaciones del Stack Tecnológico

La arquitectura técnica de **AutonoStack** ha sido concebida bajo la premisa de la **eficiencia absoluta y el mantenimiento cero (Zero-Ops)**. Para un profesional autónomo o un estudio como **bruma.studio**, la tecnología debe ser un facilitador invisible, no una fuente adicional de complejidad técnica.

En este capítulo, desglosamos el *stack* de vanguardia que sustenta la plataforma, justificando cada elección desde una perspectiva de rendimiento, seguridad y escalabilidad futura.

9.1. Arquitectura General del Sistema

AutonoStack se fundamenta en una arquitectura de **Single Page Application (SPA)** conectada a un ecosistema de **Backend as a Service (BaaS)**. Esta estructura desacoplada permite que el frontend gestione la lógica de interfaz de forma ultrarrápida en el cliente, mientras que el backend garantiza la persistencia y seguridad de los datos financieros en una infraestructura distribuida.

9.2. Tecnologías de Frontend (Capa de Presentación e Interacción)

El ecosistema de cliente de **AutonoStack** se fundamenta en un *stack* de tecnologías modernas que priorizan la modularidad, el rendimiento bruto y la accesibilidad universal. Esta elección técnica asegura que el aplicativo no solo sea funcional hoy, sino mantenible y escalable durante el crecimiento operativo de **bruma.studio**.

9.2.1. Ecosistema React: Arquitectura Basada en Componentes y Estado Reactivo

Se ha seleccionado **React.js (v18+)** como el motor principal de la interfaz debido a su paradigma de programación declarativa y su eficiente algoritmo de reconciliación mediante el **Virtual DOM**.

- **Arquitectura de Componentes Atómicos:** La interfaz se descompone en piezas de software independientes y reutilizables. Esto permite que una modificación en el diseño de un "Input de Importe" se propague instantáneamente por toda la plataforma, garantizando la consistencia visual y técnica.
- **Lógica Encapsulada en Custom Hooks:** Siguiendo las mejores prácticas de ingeniería, se ha desacoplado la lógica de negocio de la representación visual. Mediante el uso de *Hooks* personalizados, el sistema gestiona la complejidad fiscal de forma aislada.

Tabla 9.1: Catálogo de Custom Hooks y Lógica Financiera

Hook Personalizado	Responsabilidad Técnica	Objetivo de Negocio
useTaxLogic	Cálculo dinámico de bases imponibles y cuotas de IVA/IRPF.	Garantizar la precisión matemática en cada registro de factura.
useProjectStats	Agregación de datos en memoria para el cálculo de márgenes netos.	Proporcionar analítica de rentabilidad instantánea por cliente.
useAuthSession	Gestión del estado de autenticación y persistencia de sesión JWT.	Blindar el acceso a los datos financieros del usuario.

useStorage	Orquestación de subidas de archivos y generación de URLs firmadas.	Gestión eficiente de los justificantes digitales en la nube.

9.2.2. Tailwind CSS: Estilado Atómico y Sistemas de Diseño Programáticos

A diferencia de las librerías de componentes tradicionales que imponen una estética rígida, **Tailwind CSS** proporciona un sistema de utilidades de bajo nivel que ha permitido construir la **"Arquitectura de Control"** de AutonoStack desde cero.

- **Optimización del Bundle (Purge CSS):** El sistema analiza el código fuente y elimina cualquier regla CSS que no se utilice en producción. Esto resulta en archivos de estilos de apenas unos pocos KB, lo que es crítico para el cumplimiento del **RNF-02** (carga < 2s).
- **Design System a medida:** Se ha configurado un archivo tailwind.config.js que define la paleta cromática corporativa (**Verde Digital, Azul Profundo**) y la escala tipográfica, asegurando que cada componente respete el manual de identidad de la marca sin excepciones técnicas.

9.2.3. Ecosistema de Visualización de Datos

La arquitectura de visualización se fundamenta en la capacidad de transformar registros atómicos de la base de datos PostgreSQL en información estratégica en tiempo real. Esto permite al profesional independiente visualizar su salud financiera de forma instantánea.

Tremor: La Librería de Dashboards para Consultoría

Tremor se ha seleccionado como el componente principal para la creación de los indicadores clave de rendimiento (KPIs) debido a su integración nativa con **Tailwind CSS**.

- **KPI Cards**: Se utiliza para mostrar métricas críticas como el saldo actual, el IVA acumulado y la previsión de ingresos.

- **Enfoque Analítico**: A diferencia de otras librerías generales, Tremor está diseñada específicamente para interfaces de administración y cuadros de mando.
- **Componentes de Bajo Nivel**: Facilita la creación de "Donut Charts" para la categorización de gastos y barras de progreso para el seguimiento de márgenes de proyectos.

Recharts: Visualización Componible y Reactiva

Para las gráficas de tendencias y flujos de caja (Cashflow), **Recharts** aporta la flexibilidad necesaria gracias a su arquitectura basada en componentes de **React**.

- **Gráficas de Áreas**: Ideales para mapear la evolución temporal de ingresos frente a gastos desgravables.
- **Interactividad**: Permite el uso de *Tooltips* y *Zooms* dinámicos que no penalizan el rendimiento de la **SPA**.
- **Renderizado SVG**: Garantiza que las gráficas mantengan una nitidez absoluta en pantallas de alta resolución, esencial para presentaciones a clientes de **bruma.studio**.

Matriz de Justificación Técnica: Tremor vs. Recharts

Característica	Tremor	Recharts
Uso Principal	Tarjetas de métricas y layouts de dashboard.	Gráficas de ejes (Líneas, Áreas, Barras).
Estilado	Basado en utilidades de **Tailwind CSS**.	Basado en componentes declarativos de **React**.

Propósito en AutonoStack	Visualización rápida de balances y estados.	Análisis de tendencias y rentabilidad histórica.
Tecnología Base	Componentes de React + Tailwind.	SVG renderizado por React.

Integración con la Lógica de Negocio

Ambas librerías consumen los datos procesados por los *Custom Hooks* de la aplicación, traduciendo fórmulas complejas en elementos visuales comprensibles. Por ejemplo, el cálculo del margen de beneficio se representa visualmente para que el usuario identifique fugas de capital de forma inmediata:

Nota de Ingeniería: El uso de **TypeScript** asegura que los datos inyectados en estas gráficas mantengan una estructura estricta (JSON), evitando errores de renderizado ante valores nulos o inconsistentes provenientes de la base de datos.

Tabla 9.2: Matriz de Componentes Visuales y Fuentes de Datos

Componente UI	Librería Base	Fuente de Datos (Query)	Función Analítica
AreaChart de Tesorería	Recharts	transactions (agrupado por mes)	Visualizar la tendencia de liquidez y *burn rate*.

Donut de Gastos	Tremor	transactions (agrupado por categoría)	Identificar las fugas de capital y costes operativos.
ProgressBars de Margen	Tailwind/Custom	vw_rentabilidad_proyectos	Comparar la rentabilidad real frente a la estimada.
Badge de Estado IVA	Tremor	useTaxLogic (valor calculado)	Alertar sobre la carga impositiva trimestral acumulada.

9.3. Gestión del Estado Global y Sincronización Asíncrona

La orquestación técnica de **AutonoStack** se fundamenta en una simbiosis perfecta entre la agilidad del cliente y la robustez del servidor. No se trata simplemente de un repositorio de datos, sino de un ecosistema de **Ingeniería de Software de alta fidelidad** diseñado para garantizar que la información financiera de **bruma.studio** sea siempre accesible, íntegra y esté blindada contra cualquier latencia operativa.

La fluidez de la interfaz, factor crítico para el cumplimiento del **RNF-02 (Carga < 2s)**, depende directamente de la estrategia de gestión de estados. En **AutonoStack**, se ha implementado un paradigma de **Estado Híbrido** que diferencia radicalmente entre los datos efímeros de la interfaz y la verdad persistente del servidor.

9.3.1. Estado de Servidor: El Paradigma "Stale-While-Revalidate"

Para manejar la asincronía de la base de datos de Supabase sin sacrificar la experiencia de usuario, se utiliza **React Query / SWR**. Esta tecnología permite que la aplicación sea "consciente" del tiempo de vida del dato.

- **Persistencia en Caché Local:** Cuando el usuario consulta su dashboard, los datos se almacenan en una memoria caché de alta velocidad. Al navegar de vuelta a la pantalla, la información se muestra de forma instantánea ($0ms$ de latencia percibida), mientras un proceso en segundo plano verifica en PostgreSQL si existen actualizaciones.
- **Actualizaciones Optimistas (Optimistic Updates):** En procesos críticos como el registro de un gasto, la aplicación "asume" el éxito de la operación y actualiza los gráficos e indicadores inmediatamente. Si el servidor devuelve un error, se realiza un *rollback* automático. Esto elimina la necesidad de los tradicionales "spinners" de carga que interrumpen el flujo cognitivo del profesional.

9.3.2. Estado Local: Coherencia de Interfaz vía Context API

Para los datos que no requieren persistencia en base de datos pero sí consistencia en la navegación, se emplea **React Context**.

- **Configuración de Sesión:** Mantiene los filtros activos del libro diario, la visibilidad de los balances y el estado del "Modo Demo" a través de todas las vistas del aplicativo.

Tabla 9.3: Matriz Comparativa de Gestión de Estados

Dimensión de Datos	Tecnología Empleada	Fuente de Verdad	Estrategia de Sincronización
Transacciones y Proyectos	**React Query / SWR**	PostgreSQL (Remoto)	Sincronización asíncrona con revalidación en segundo plano.
Preferencias de UI / Filtros	**React Context**	Memoria RAM (Local)	Reactividad inmediata ante la interacción del usuario.
Justificantes Binarios	**Supabase SDK**	Storage Buckets	*Lazy loading* (carga bajo demanda) para optimizar ancho de banda.
Sesión de Seguridad	**Supabase Auth**	LocalStorage / JWT	Validación persistente mediante tokens de acceso cifrados.

9.4. Backend e Infraestructura (Capa de Servicios)

La infraestructura de **AutonoStack** ha sido diseñada bajo el concepto de **Serverless de Alta Disponibilidad**. Al delegar la gestión de máquinas y servidores en **Supabase**, el proyecto minimiza los riesgos de caída del sistema y los costes de mantenimiento, permitiendo que el 100% de los recursos se centren en la lógica de negocio fiscal.

9.4.1. Infraestructura BaaS: Supabase como Núcleo Operativo

Supabase se ha consolidado como la alternativa de código abierto a Firebase más robusta del mercado, proporcionando un ecosistema de herramientas industriales integradas nativamente.

Tabla 9.4: Desglose de Servicios de Infraestructura

Servicio	Componente Técnico	Impacto en el Proyecto	Justificación de Ingeniería
Autenticación	**Supabase Auth (GoTrue)**	Gestión de Identidad	Implementa el estándar OAuth2 y JWT, permitiendo sesiones seguras y escalables.
Base de Datos	**PostgreSQL 15**	Persistencia Relacional	Motor de base de datos más avanzado del mundo, garantizando integridad atómica.
Almacenamiento	**Supabase Storage**	Gestión Documental	Repositorio de objetos con CDN integrado para la entrega rápida de justificantes.
Seguridad de Capa	**PostgREST**	Generación de API	Transforma el esquema de la DB en una API RESTful instantánea y segura.

Lógica Serverless	**Edge Functions**	Procesamiento	Ejecución de lógica pesada (ej. generación de PDFs) cerca del usuario final.

9.4.2. Motor de Datos: PostgreSQL y la Soberanía Financiera

A diferencia de las arquitecturas NoSQL (como MongoDB), **PostgreSQL** es el estándar de oro para aplicaciones contables. Su capacidad para manejar transacciones **ACID** (Atomicidad, Consistencia, Aislamiento y Durabilidad) asegura que nunca se pierda un céntimo de la contabilidad de **bruma.studio**.

- **Integridad Referencial:** Mediante claves foráneas, el sistema garantiza que no existan facturas sin un proyecto asociado o gastos sin un usuario propietario.
- **Vistas y Funciones SQL:** La lógica compleja de agregación (como el cálculo del margen de beneficio neto) se ejecuta directamente en el motor de la base de datos, lo que resulta en una velocidad de respuesta órdenes de magnitud superior al procesamiento en el frontend.

Tabla 9.5: Cumplimiento de Propiedades ACID en AutonoStack

Propiedad ACID	**Implementación en PostgreSQL**	**Garantía para el Autónomo**
Atomicidad	Transacciones BEGIN / COMMIT.	Una factura se guarda completa o no se guarda nada (evita datos corruptos).

Consistencia	Restricciones CHECK y NOT NULL.	Los importes negativos o fechas inválidas son rechazados por el motor de datos.
Aislamiento	Niveles de aislamiento de transacciones.	Dos operaciones simultáneas no interfieren entre sí, evitando duplicidades.
Durabilidad	Escritura en registros WAL (Write-Ahead Logging).	Ante un fallo eléctrico en el centro de datos, los registros confirmados no se pierden.

9.5. Infraestructura de Despliegue y CI/CD: Vercel

El despliegue final de **AutonoStack** se orquesta a través de **Vercel**, la plataforma de computación en la nube líder para ecosistemas React.

- **Global Edge Network:** El código del frontend se distribuye en nodos geográficamente cercanos a **bruma.studio**, reduciendo el tiempo de respuesta inicial.
- **Pipeline de Integración Continua:** Cada cambio en el repositorio de GitHub es auditado automáticamente; si el código pasa las pruebas de calidad, se despliega en producción en cuestión de segundos, permitiendo actualizaciones constantes sin interrupción del servicio.

9.4. Ecosistema de Desarrollo y DevOps: La Ingeniería del Ciclo de Vida

La excelencia operativa de **AutonoStack** no reside únicamente en la elegancia de su código fuente, sino en una infraestructura de **Ingeniería de Ciclo de Vida (DevOps)** de alta fidelidad que garantiza evolución constante, seguridad impenetrable y disponibilidad global. Para un entorno de consultoría profesional como el de **bruma.studio**, el despliegue de software ha dejado de ser un evento puntual para transformarse en un flujo continuo de valor, blindado por herramientas de auditoría estática y sistemas de automatización de vanguardia.

9.4.1. Despliegue e Integración Continua (CI/CD): Flujo de Entrega Ininterrumpida

El proceso de **Integración Continua (CI)** y **Despliegue Continuo (CD)** constituye la columna vertebral que permite a la plataforma mantenerse en un estado de "producción permanente" sin tiempos de inactividad.

- **GitHub como Centro de Orquestación:** Actúa como el repositorio maestro y el cerebro de la automatización. Mediante el uso de **GitHub Actions**, cada actualización de código dispara un flujo de trabajo que verifica la integridad del aplicativo antes de permitir su fusión con la rama principal.
- **Vercel Edge Network y Despliegue Atómico:** Se ha seleccionado **Vercel** debido a su arquitectura distribuida. Vercel despliega el frontend de AutonoStack en una red global de nodos de proximidad (Edge), reduciendo la latencia de acceso a niveles de milisegundos. Cada despliegue es "inmutable", lo que permite realizar un *rollback* (reversión) instantáneo a una versión estable en caso de anomalía.

Tabla 9.6: Matriz del Ciclo de Vida de Despliegue (Pipeline)

Etapa del Flujo	Tecnología	Acción de Ingeniería	Objetivo de Calidad
Control de Versiones	**GitHub**	Gestión de ramas (*Branching*) y *Peer Review*.	Garantizar que el código sea auditado por humanos antes de automatizarse.
Auditoría Estática	**ESLint / Prettier**	Análisis automático de sintaxis y estilo.	Eliminar deudas técnicas y mantener la legibilidad profesional.
Integración (CI)	**GitHub Actions**	Ejecución de *scripts* de compilación y pruebas.	Validar que los nuevos cambios no rompan funcionalidades existentes.
Build y Optimización	**Next.js Engine**	Minificación de activos (JS, CSS, Imágenes).	Maximizar el rendimiento de carga y cumplimiento del RNF-02.
Despliegue (CD)	**Vercel Edge**	Generación de URL de previsualización inmutable.	Permitir pruebas en entornos de *staging* idénticos a producción.

9.4.2. Estrategia de Pruebas Automatizadas (QA)

Siguiendo la estructura de **myPocket**, se ha implementado una jerarquía de pruebas para asegurar que los cálculos financieros (como el IVA y el margen neto) sean exactos y no sufran regresiones durante el desarrollo.

- **Unit Testing (Pruebas Unitarias):** Enfoque en las funciones de lógica fiscal. Cada algoritmo de cálculo de impuestos se prueba de forma aislada para garantizar precisión matemática absoluta.
- **Integration Testing (Pruebas de Integración):** Validación de la comunicación entre los componentes de React y la API de Supabase, asegurando que el flujo de datos sea consistente.

Tabla 9.7: Matriz de Cobertura de Pruebas

Nivel de Prueba	Herramienta	Ámbito de Aplicación	Garantía para el Negocio
Unitaria	**Jest**	Funciones de cálculo impositivo y *helpers*.	Precisión decimal en balances e informes fiscales.
Componentes	**React Testing Lib.**	Widgets del Dashboard y formularios CRUD.	Usabilidad y respuesta correcta de la interfaz.
E2E (Punta a Punta)	**Playwright**	Flujo completo: Login ➔ Registro de Gasto.	Confirmación de que los flujos críticos son funcionales.

9.4.3. Herramientas de Calidad, Tipado y Blindaje de Código

Para que una herramienta financiera sea fiable, el código debe ser predecible. AutonoStack implementa una capa de seguridad en tiempo de desarrollo que previene el 90% de los errores lógicos antes de su despliegue.

- **TypeScript (El Contrato de Datos):** Define interfaces estrictas para entidades como Transaction o Project. Esto impide, por ejemplo, que un desarrollador intente registrar un gasto sin un user_id válido, blindando la integridad de la base de datos desde el origen.
- **Gestión de Secretos y Seguridad:** Las claves de API de Supabase y variables de entorno sensibles nunca se almacenan en el repositorio. Se gestionan mediante el sistema de **Environment Variables** cifrado de Vercel y GitHub Secrets.

Tabla 9.8: Ecosistema de Blindaje del Código

Herramienta	Función Técnica	Impacto en la Aplicación
TypeScript	Tipado estático y estructural.	Prevención de errores de tipo en cálculos matemáticos.
ESLint	Linter de código estático.	Detección de patrones inseguros de código.
Husky	*Git Hooks* (Pre-commit).	Impide subir código si no pasa los tests y el linter.
Dependabot	Auditoría de dependencias.	Actualización automática de librerías ante vulnerabilidades.

9.5. Resumen de Ventajas del Ecosistema DevOps

La integración de este ecosistema proporciona a **AutonoStack** tres ventajas competitivas inalcanzables para desarrollos convencionales:

1. **Time-to-Market Acelerado:** Capacidad de desplegar nuevas funcionalidades de forma segura y automática en minutos, respondiendo rápidamente a cambios normativos.
2. **Seguridad por Defecto:** El uso de **TypeScript** y auditorías de vulnerabilidades garantiza que la lógica de seguridad se implemente siempre bajo estándares de "Confianza Cero".
3. **Mantenibilidad de Consultoría:** Un código limpio, testado y tipado es un activo financiero; permite que la plataforma crezca orgánicamente sin que la complejidad técnica se vuelva inmanejable para **bruma.studio**.

9.5. Matriz de Justificación Técnica (Análisis Comparativo de Stack)

A continuación, se presenta la matriz de decisión que fundamenta la arquitectura del aplicativo. Esta comparativa técnica demuestra por qué el *stack* elegido es la solución óptima para un entorno de consultoría profesional de alta disponibilidad.

Tabla 9.8: Matriz de Decisiones Arquitectónicas y Justificación de Ingeniería

Categoría Técnica	Opción Seleccionada	Alternativa Evaluada	Razón Técnica del Descarte	Ventaja Estratégica para AutonoStack
Framework de Interfaz	React.js	Angular / Vue.js	Curva de aprendizaje más pronunciada y menor ecosistema de librerías para dashboards financieros.	Arquitectura de componentes atómicos y renderizado eficiente mediante Virtual DOM.
Arquitectura de Estilos	Tailwind CSS	Bootstrap / Sass	Generación de archivos CSS voluminosos y dependencia de estilos predefinidos rígidos.	Estilado atómico con PurgeCSS, reduciendo el peso del bundle en un 70% frente a Bootstrap.

Infraestructura Backend	Supabase (BaaS)	Node.js + Express (Custom)	Elevado tiempo de desarrollo inicial en gestión de sockets, auth y APIs (aprox. $+60\%$ tiempo).	Implementación instantánea de PostgREST y GoTrue para autenticación segura.
Motor de Persistencia	PostgreSQL	MongoDB (NoSQL)	Carencia de integridad referencial nativa y riesgos en la consistencia de transacciones financieras.	Soporte nativo para transacciones ACID y políticas de seguridad profunda (RLS).
Gestión de Archivos	Supabase Storage	AWS S3 / Firebase Storage	Complejidad excesiva en la configuración de permisos y costes ocultos por	Integración nativa con la base de datos para asegurar el blindaje de facturas y tickets.

			transferencia de datos.	
Hosting y Despliegue	Vercel	Heroku / DigitalOcean	Necesidad de configuración manual de pipelines CI/CD y mayor latencia en la red de entrega (CDN).	Red global de Edge Nodes que garantiza una carga instantánea desde cualquier ubicación.

9.6. Análisis Profundo de las Elecciones Estratégicas

9.6.1. Superioridad Relacional: PostgreSQL frente al Paradigma NoSQL

Para una plataforma que gestiona obligaciones fiscales, la integridad de los datos es innegociable. Mientras que las bases de datos NoSQL (como MongoDB) priorizan la flexibilidad del esquema, PostgreSQL garantiza que no existan inconsistencias matemáticas en el cálculo de impuestos.

- **Criterio de Fallo:** Un error de esquema en NoSQL podría permitir facturas sin tax_rate, invalidando un trimestre fiscal completo. En PostgreSQL, las

restricciones NOT NULL y CHECK actúan como un cortafuegos preventivo a nivel de bit.

9.6.2. Eficiencia de Desarrollo: El impacto del Backend-as-a-Service (BaaS)

La adopción de Supabase permite que la lógica de desarrollo se centre en un 90% en la experiencia del usuario y en la precisión de los cálculos de bruma.studio, delegando la gestión de la infraestructura (parches de seguridad, escalado de base de datos, optimización de APIs) en servicios gestionados de alto rendimiento. Esta decisión técnica permite que el proyecto pase de la fase de prototipo a producción en un tiempo récord sin sacrificar la seguridad industrial.

9.6.3. El Ecosistema React y la Inteligencia Visual

La elección de React no solo facilita la creación de componentes reutilizables, sino que abre la puerta a librerías de visualización de datos de primer nivel como Tremor y Recharts. Estas librerías están optimizadas para el ciclo de vida de React, permitiendo que las gráficas de flujo de caja se re-rendericen solo cuando los datos cambian, manteniendo una fluidez constante incluso en dispositivos móviles de gama media.

9.7. Análisis de Eficiencia y Rendimiento Esperado (Performance Metrics)

Gracias a la sinergia entre Tailwind CSS y Vercel, el aplicativo está diseñado para superar los estándares de Google Lighthouse en sus métricas de rendimiento.

Tabla 9.9: Objetivos de Rendimiento Técnico (SLAs)

Métrica de Rendimiento	Valor Objetivo	Tecnología Responsable
First Contentful Paint (FCP)	< 0.8s	Vercel Edge Cache / Tailwind CSS
Time to Interactive (TTI)	< 1.5s	React Optimized Bundle
Disponibilidad del Servicio (Uptime)	99.99%	Supabase / Vercel Infrastructure
Integridad de Datos	100%	PostgreSQL ACID Transactions

La viabilidad económica de **AutonoStack** no es un factor accidental, sino el resultado de una estrategia de **Ingeniería de Costes** diseñada para maximizar la rentabilidad de **bruma.studio**. En un sector donde los costes de infraestructura pueden canibalizar los márgenes de beneficio del profesional independiente, se ha optado por un ecosistema de **Sostenibilidad Financiera Extrema**, basado en el aprovechamiento de las capas gratuitas de nivel industrial (*Free Tiers*) de los proveedores líderes del mercado.

9.8. Análisis de Costes e Infraestructura

9.8.1. Modelo de Inversión Cero (Zero-Cost Infrastructure)

Para el volumen operativo actual y proyectado de un estudio creativo, el *stack* tecnológico de AutonoStack opera bajo un **modelo de coste operativo de $0€$ mensuales**. Esta eficiencia se logra mediante la orquestación de servicios *Serverless* que solo consumen recursos bajo demanda, eliminando los gastos fijos de mantenimiento de servidores dedicados.

Tabla 9.10: Desglose de Capacidades y Límites del Tier Gratuito

Proveedor	Servicio Incluido	Límite del Tier Gratuito	Valor Económico Estimado
Supabase	Base de Datos (PostgreSQL)	500MB de almacenamiento / 5GB ancho de banda.	approx 25€/mes
Supabase	Auth & Seguridad	Usuarios ilimitados / 50.000 MAU (Activos Mes).	approx 15€/mes
Supabase	Storage (Justificantes)	1GB de almacenamiento de archivos.	approx 5€/mes
Vercel	Hosting & CDN	Ancho de banda ilimitado (uso personal/proyectos).	approx 20€/mes
Vercel	Funciones Edge	100.000 ejecuciones mensuales.	approx 10€/mes

TOTAL	Infraestructura Global	Capacidad Profesional Completa	Ahorro: approx 75€/mes

9.8.2. Seguridad de Grado Industrial sin Sobrecoste

Es un error común asociar el "coste cero" con una merma en la seguridad. AutonoStack hereda los protocolos de protección de datos de los grandes centros de datos mundiales (AWS y GCP) sobre los que corren Supabase y Vercel, garantizando que la información de **bruma.studio** esté blindada bajo estándares bancarios.

Tabla 9.11: Estándares de Seguridad y Privacidad Integrados

Capa de Seguridad	**Tecnología / Protocolo**	**Garantía Técnica**
Cifrado en Tránsito	SSL / TLS 1.3	Encriptación de toda la comunicación navegador-servidor.
Cifrado en Reposo	AES-256	Los datos en PostgreSQL y archivos en Storage están cifrados físicamente.
Aislamiento	RLS (Row Level Security)	Seguridad a nivel de bit: un usuario es invisible para otro en la misma DB.
Autenticación	JWT (JSON Web Tokens)	Sesiones firmadas digitalmente que caducan automáticamente.
Infraestructura	SOC2 / ISO 27001	Cumplimiento de normativas internacionales de seguridad de la información.

9.8.3. Métricas de Rendimiento: La Experiencia de Carga Instantánea

La combinación de una base de datos optimizada en **Supabase** y una red de entrega de contenido (CDN) global en **Vercel** permite que AutonoStack cumpla con los requisitos de **Eficiencia Visual** más exigentes. La latencia de red se reduce al mínimo teórico, permitiendo que el autónomo interactúe con sus finanzas sin fricciones.

Métrica Crítica: El **Time to First Byte (TTFB)** se mantiene consistentemente por debajo de los **200ms**, lo que garantiza que el dashboard se perciba como una aplicación nativa instalada en el dispositivo, no como una web convencional.

Tabla 9.12: KPIs de Rendimiento de Infraestructura (SLA Proyectado)

Métrica de Rendimiento	Objetivo (SLA)	Impacto en la Experiencia (UX)
Uptime (Disponibilidad)	99.9%	La plataforma está disponible 24/7 para consultas de última hora.
Latencia de API	< 150ms	Las operaciones de guardado de facturas son instantáneas.
LCP (Largest Contentful Paint)	< 1.2s	El dashboard visual está listo para interactuar en un segundo.
Tasa de Error de Red	< 0.01%	Fiabilidad absoluta en la subida de documentos pesados (PDF).

9.8.4. Escalabilidad y Roadmap de Crecimiento

Si bien el modelo actual es de coste cero, la arquitectura está preparada para escalar junto con el éxito de **bruma.studio**. El paso a un modelo de pago (*Pro Plan*)

es transparente y no requiere cambios en el código, permitiendo ampliar los límites de almacenamiento de forma elástica si el volumen de facturación superase los miles de registros mensuales.

- **Escalabilidad Vertical:** Capacidad para manejar bases de datos de hasta varios Terabytes.
- **Escalabilidad Horizontal:** Distribución de archivos en múltiples regiones geográficas para optimizar el acceso internacional.

10. Metodología de Desarrollo y Planificación

La ejecución de **AutonoStack** no se ha planteado como un simple ejercicio de programación, sino como una **obra de ingeniería de software** orquestada bajo los más altos estándares de rigor profesional. Para transformar una necesidad administrativa crítica en una solución tecnológica de alta disponibilidad, se ha implementado una metodología de vanguardia que garantiza la convergencia entre la visión estratégica y la solvencia técnica.

A continuación, se detalla el marco metodológico, la planificación temporal y la gestión de riesgos que han permitido la entrega de este ecosistema Full Stack con una precisión quirúrgica.

10.1. Marco Metodológico: El Paradigma del Desarrollo Ágil (Agile Scrum)

El desarrollo de AutonoStack se ha regido por el marco de trabajo **Scrum**, adaptado para maximizar la entrega de valor incremental y la capacidad de respuesta ante requisitos volátiles. No se ha optado por un modelo en cascada tradicional,

propenso a desviaciones presupuestarias y obsolescencia funcional; en su lugar, se ha implementado un flujo iterativo que permite que la arquitectura de datos y la experiencia de usuario evolucionen en una simbiosis técnica constante.

10.1.1. Artefactos y Ceremonias del Sistema

Para asegurar que el ritmo de desarrollo fuera sostenible y transparente, se han integrado las ceremonias clásicas de Scrum con un enfoque de ingeniería financiera:

- **Sprint Planning:** Sesiones de inicio de ciclo donde se desglosaron las épicas (como "Gestión de Impuestos") en tareas técnicas atómicas dentro del *Product Backlog*.
- **Daily Sync:** Reuniones de sincronización diaria de 15 minutos para identificar bloqueos técnicos, especialmente en la integración de las políticas **RLS** y los *hooks* de React.
- **Sprint Review & Demo:** Presentaciones funcionales al finalizar cada ciclo para validar la precisión de las gráficas de *cashflow* frente a datos reales de **bruma.studio**.
- **Definition of Done (DoD):** Criterio de "Terminado" riguroso que incluía: código tipado con TypeScript, superación de tests unitarios de lógica fiscal y cumplimiento de accesibilidad **WCAG 2.1**.

Tabla 10.1: Pilares del Marco Ágil en AutonoStack

Pilar Metodológico	Implementación en la Consultoría	Impacto en el Producto Final

Iteratividad Crítica	Segmentación del proyecto en Sprints de 2 semanas de duración.	Validación constante de la lógica fiscal y las gráficas dinámicas.
Transparencia Operativa	Uso de tableros Kanban para la trazabilidad total de cada tarea.	Visibilidad absoluta sobre el progreso y eliminación de cuellos de botella.
Inspección y Adaptación	Revisiones técnicas diarias para asegurar la integridad del código.	Detección precoz de inconsistencias en el modelo relacional de PostgreSQL.
Entregables Funcionales	Generación de incrementos de software operativos tras cada ciclo.	Disponibilidad de un **MVP** funcional desde la semana 4.

10.2. Ciclo de Vida del Proyecto: Fases de Ingeniería

La planificación estratégica se ha estructurado en cinco fases de alta densidad técnica, diseñadas para blindar la viabilidad del aplicativo desde su concepción hasta su despliegue en producción.

- **Fase 1: Inmersión, Discovery y Análisis (Semanas 1-2):** Fase dedicada a la disección de los flujos de trabajo de bruma.studio. Se ejecutó el levantamiento de requisitos (RF y RNF) y se definió la Matriz de Trazabilidad. El entregable clave fue el diseño del Modelo Entidad-Relación (E-R).

- **Fase 2: Arquitectura y Diseño de Experiencia UX/UI (Semanas 3-4):** Conceptualización del Design System y prototipos de alta fidelidad en Figma, validando la ergonomía de la interfaz y la jerarquía visual del dashboard.
- **Fase 3: Desarrollo del Núcleo Full Stack (Semanas 5-8):** Configuración de tablas, tipos enumerados y políticas RLS en Supabase, junto a la implementación de componentes reactivos y gestión de estados con SWR.
- **Fase 4: QA, Seguridad y Validación Fiscal (Semanas 9-10):** Sometimiento a pruebas de estrés y validación matemática del IVA trimestral frente a modelos oficiales de la AEAT.
- **Fase 5: Despliegue, DevOps y Cierre (Semanas 11-12):** Orquestación final en Vercel, traspaso de credenciales y entrega del manual de usuario.

10.3. Cronograma y Desglose Detallado de Actividades

La ejecución de AutonoStack se ha estructurado como una hoja de ruta crítica de dieciocho semanas, diseñada para armonizar los requisitos académicos con las demandas técnicas reales.

Fase I: Definición y Diseño Estratégico (Septiembre - Octubre)

- **12 de septiembre:** Sesión de apertura y presentación formal del núcleo del proyecto.
- **19 de septiembre:** Fase de **Branding**: Creación de la imagen corporativa de la empresa encargada del desarrollo.
- **26 de septiembre:** Elaboración del contrato de prestación de servicios y ejecución de la sesión de *Discovery* para recogida de necesidades.
- **03 de octubre:** Definición técnica de los Requisitos Funcionales y No Funcionales.

- **10 de octubre (Hito de Diseño):** Fase de **Prototipado de Alta Fidelidad**: Diseño de interfaces en Figma y establecimiento del flujo de navegación.

Fase II: Arquitectura de Datos y Validación Técnica (Octubre - Noviembre)

- **17 de octubre:** Desarrollo de la estructura técnica de la base de datos en Supabase.
- **24 de octubre:** Creación y normalización del **Modelo Relacional** en PostgreSQL, asegurando la integridad referencial.
- **31 de octubre:** Sesión de **Aceptación de Diseño**: Presentación a la empresa de las interfaces y la base de datos para su validación definitiva.
- **07 de noviembre:** Congelación del **Stack Tecnológico**: Decisión firme de utilizar React, Tailwind y Supabase.

Fase III: Desarrollo de Documentación y Lógica de Negocio (Noviembre - Diciembre)

- **14 de noviembre:** Estructuración inicial de toda la documentación y repositorio central.
- **21 de noviembre:** Definición de la arquitectura de contenidos para los manuales de usuario y técnicos.
- **05 de diciembre:** Desarrollo de la primera versión funcional de la documentación de soporte técnico.

Fase IV: Testing, Despliegue y Producción (Diciembre - Enero)

- **12 de diciembre:** Análisis de infraestructura y evaluación de opciones de despliegue Cloud.
- **19 de diciembre: Testing en Local**: Ejecución de pruebas de compatibilidad en entorno local (TomCat).
- **09 de enero: Hito de Producción**: Despliegue oficial y lanzamiento en la red global de **Vercel**.
- **16 de enero:** Fase de ajustes finales: Corrección de errores (*Bug fixing*) y optimización del rendimiento.
- **23 de enero:** Cierre definitivo de la memoria técnica y los manuales finales.
- **30 de enero: Empaquetado y Cierre**: Preparación de la defensa del proyecto ante el tribunal.

10.4. Análisis de la Ruta Crítica (Critical Path Method)

Para asegurar la entrega el 30 de enero, se identificaron las dependencias técnicas que constituían la ruta crítica del proyecto:

- **Dependencia Datos-Lógica:** Era imperativo finalizar el modelo relacional (24 oct) antes de iniciar la programación de los *Custom Hooks* financieros.
- **Dependencia Diseño-Frontend:** El prototipado del 10 de octubre fue el requisito previo para la implementación de los componentes en Tailwind CSS.
- **Dependencia Seguridad-Producción:** La configuración de las políticas **RLS** fue el "cuello de botella" necesario para garantizar un despliegue seguro en Vercel el 09 de enero.

10.5. Gestión de Tareas y Flujo de Trabajo (Workflow)

El desarrollo se gestionó mediante una metodología mixta que combinó el rigor de **Git** con la agilidad visual:

- **Gestión de Tareas (Kanban):** Se utilizó **GitHub Projects** para desglosar cada hito en tareas atómicas (*Issues*), clasificándolas en: *To Do, In Progress, Review* y *Done*.
- **Flujo de Trabajo Git (GitFlow):**
  - main: Solo código en producción (desplegado en Vercel).
  - develop: Rama de integración de nuevas funcionalidades.
  - feature/*: Desarrollo aislado de módulos específicos (ej. feature/dashboard).

10.6. Diseño y Prototipado: Del Concepto al Píxel

El hito del **10 de octubre** marcó un punto de inflexión. El proceso de diseño se dividió en tres niveles de abstracción:

1. **Wireframes (Lo-Fi):** Esbozos estructurales para validar la ubicación de los KPIs en el dashboard.
2. **Prototipo Interactivo (Hi-Fi):** Creación de componentes reales en Figma, aplicando el sistema de espaciados de 8px y la semántica de colores definida para bruma.studio.
3. **Handoff técnico:** Traducción de los diseños a variables de **Tailwind CSS**, permitiendo una transición fluida al desarrollo iniciado en noviembre.

10.7. Matriz Resumen de Hitos Críticos

Fecha Clave	Hito Alcanzado	Entregable Asociado	Estado
03-Oct	Cierre de Requisitos	Documento de Especificación RF/RNF	Finalizado
31-Oct	Validación de Diseño	Prototipo Figma y Esquema SQL	Finalizado
07-Nov	Congelación de Stack	Informe de Selección Tecnológica	Finalizado
09-Ene	Despliegue Cloud	Aplicativo en Producción (Vercel)	Finalizado
23-Ene	Cierre Documental	Manuales de Usuario y Técnico Finales	Finalizado
30-Ene	Entrega Final	Software Empaquetado y Defensa	**Completado**

10.8. Justificación de la Planificación

Este cronograma ha permitido que el desarrollo de AutonoStack se realice de forma controlada, minimizando el riesgo de "Scope Creep". La separación entre la fase de base de datos (octubre) y la de despliegue (enero) garantizó que la lógica de cálculo fuera validada matemáticamente antes de ser visualizada en el dashboard final.

10.9. Gestión de Riesgos y Mitigación Estratégica

En el desarrollo de una solución financiera, la improvisación es el mayor riesgo. Para asegurar la integridad de los datos de **bruma.studio**, se ha diseñado la siguiente matriz:

Tabla 10.3: Matriz de Gestión de Riesgos Tecnológicos

Riesgo Identificado	Gravedad	Probabilidad	Estrategia de Mitigación de Ingeniería
Inconsistencia de Cálculos	Crítica	Muy Baja	Pruebas unitarias de lógica fiscal y restricciones CHECK en PostgreSQL.
Fuga de Información	Crítica	Baja	Activación de **Row Level Security (RLS)** en Supabase y gestión de secretos.
Latencia de Interfaz	Media	Media	Gestión de caché mediante **SWR** y componentes de alta densidad (**Tremor**).
Desviación de Plazos	Baja	Alta	Seguimiento milimétrico del cronograma semanal definido en el repositorio.

10.10. Ecosistema de Herramientas de Planificación y Control

Se ha utilizado un *stack* de gestión de primer nivel para asegurar una trazabilidad del 100% de los procesos:

- **Gestión de Producto: GitHub Projects** para la orquestación de tareas semanales mediante metodología Kanban.
- **Control de Versiones:** El uso de **Git y GitHub** ha facilitado un histórico de cambios auditable y reversible.
- **Diseño: Figma** para el desarrollo de interfaces de alta fidelidad presentadas el 10 de octubre.
- **Documentación:** Uso de **Markdown** para mantener la documentación técnica (README) sincronizada con el código.

10.11. Resumen de la Lógica de Negocio e Inteligencia Estratégica

A lo largo de los hitos establecidos, la prioridad ha sido dotar al autónomo de soberanía financiera. El epicentro es el **Algoritmo de Rentabilidad Neta** integrado en el Dashboard el 5 de diciembre.

Esta fórmula transforma el registro contable tradicional en una herramienta de decisión estratégica, permitiendo identificar los proyectos de mayor retorno en tiempo real.

10.12. Gestión de Recursos y Capacidades

El desarrollo de **AutonoStack** ha sido ejecutado bajo un modelo de **Recursos Optimizados**, donde la figura del desarrollador asume un rol *Full-Stack* con soberanía total sobre el ciclo de vida del software.

- **Recursos Humanos:** Coordinación técnica liderada por Lucía Fernández Florencio.
- **Recursos Tecnológicos:** Uso intensivo de herramientas *Cloud* (Supabase, Vercel) y entornos de desarrollo locales (VS Code, Vite) para minimizar la fricción operativa.
- **Recursos Documentales:** Repositorio centralizado en GitHub para garantizar la disponibilidad de la memoria técnica y el código fuente.

10.13. Aseguramiento de la Calidad y Control (QA)

Como cierre de la metodología, se ha implementado un protocolo de control de calidad basado en la **Revisión Continua**.

- **Validación de Requisitos:** Contraste sistemático de cada funcionalidad con la matriz de Requisitos Funcionales y No Funcionales definida el 03 de octubre.
- **Pruebas de Despliegue:** Ejecución de pruebas en entorno local (TomCat) el 19 de diciembre antes del paso definitivo a la red Edge de Vercel el 09 de enero.
- **Refinamiento de Bugs:** Fase específica de corrección y optimización (16 de enero) para garantizar una experiencia de usuario libre de fricciones técnicas.

10.9. Gestión de Riesgos y Planes de Contingencia

En el desarrollo de una solución de soberanía financiera como **AutonoStack**, el riesgo no es sólo técnico, sino reputacional y legal. Para blindar la entrega del **30 de enero**, se ha diseñado una matriz que no solo identifica el peligro, sino que dicta la "Hoja de Ruta de Emergencia".

10.9.1. Matriz de Riesgos y Protocolos de Respuesta

Riesgo Identificado	**Gravedad**	**Estrategia de Mitigación (Prevención)**	**Plan de Contingencia (Acción si ocurre)**
Inconsistencia de Cálculos	**Crítica**	Pruebas unitarias de lógica fiscal y restricciones CHECK en PostgreSQL.	**Auditoría Manual:** Bloqueo de edición de facturas + Recálculo forzado mediante *scripts* SQL de emergencia.
Fuga de Datos Financieros	**Crítica**	Activación de **Row Level Security (RLS)** en Supabase y gestión de secretos.	**Kill-Switch:** Suspensión temporal del servicio + Rotación masiva de claves API + Notificación a **bruma.studio**.

Latencia / Caída de Vercel	Media	Uso de la red **Edge** de Vercel para distribución global.	**Failover Local:** Redirección temporal a una instancia de respaldo o despliegue en entorno local (**TomCat**).
Desviación del Cronograma	Media	Seguimiento semanal estricto según el cronograma del 12 de sept.	**Reducción de Alcance:** Priorización de funciones núcleo (CRUD) y aplazamiento de manuales secundarios (Hito 23-ene).
Incompatibilidad de Storage	Baja	Pruebas de subida de archivos PDF/JPG iniciadas el 24 de marzo.	**Bypass de Almacenamiento:** Uso temporal de almacenamiento en base de datos (Base64) hasta restaurar el Bucket.

10.10. Planes de Contingencia Operativos

Para asegurar que la transición entre el hito de **Testing en Local (19-dic)** y el de **Producción (09-ene)** fuera segura, se establecieron tres protocolos de emergencia:

Protocolo A: Recuperación de Desastres (Database)

Ante una pérdida de integridad en PostgreSQL (detectada por los disparadores de auditoría), el plan de contingencia dicta la restauración inmediata del último *Snapshot* diario de Supabase. Esto garantiza que el autónomo nunca pierda más de 24 horas de contabilidad.

Protocolo B: Contingencia de Despliegue (Vercel)

Si durante el hito de producción del **09 de enero** se detectan errores de compatibilidad en la red Cloud, se activará el plan de "Rollback Atómico". Vercel permite revertir a la versión estable anterior en menos de 10 segundos, asegurando la continuidad del servicio mientras se depuran los *bugs* en el entorno de desarrollo.

Protocolo C: Mitigación de Errores Críticos (16-ene)

Durante la fase de **Ajustes Finales (16 de enero)**, si se identifica un error que comprometa la veracidad del IVA trimestral, el aplicativo entrará en "Modo Lectura". Esto impide que el usuario tome decisiones basadas en datos erróneos mientras se aplica el parche de seguridad en la lógica de **jsPDF** o el dashboard.

10.11. Gestión de Recursos y Garantía de Calidad (QA)

Como cierre de la metodología, se ha implementado un protocolo de control basado en la **Revisión Continua** para cumplir con la entrega del software empaquetado el **30 de enero**:

- **Validación de Roles:** Verificación de que el rol de "Invitado" solo posee permisos de lectura, evitando la alteración de datos de auditoría.
- **Aseguramiento Documental:** El cierre del **23 de enero** garantiza que no existan discrepancias entre la versión final del software y las guías entregadas al usuario.
- **Testing Cruzado:** Cruce de datos entre el entorno local (TomCat) y producción (Vercel) para asegurar que las variables de entorno no alteran los resultados financieros.

11. Manual Técnico

El presente manual describe la arquitectura interna, las dependencias tecnológicas y los protocolos de despliegue de **AutonoStack**. Su objetivo es garantizar la mantenibilidad y escalabilidad del sistema, permitiendo una transición fluida entre entornos de desarrollo y producción.

11.1. Arquitectura del Sistema

AutonoStack utiliza una arquitectura desacoplada basada en el paradigma **SPA (Single Page Application)** conectada a un ecosistema **BaaS (Backend as a Service)**.

- **Capa de Presentación (Frontend):** Construida con **React** y **Vite**, gestionando el estado de forma reactiva y optimizando el renderizado mediante el Virtual DOM.
- **Capa de Servicios (BaaS):** Delegada en **Supabase**, que orquesta de forma nativa la base de datos PostgreSQL, la autenticación JWT y el almacenamiento de objetos.
- **Capa de Despliegue (Edge):** El aplicativo se sirve a través de la red global de **Vercel**, utilizando *Edge Functions* para minimizar la latencia.

11.2. Entorno de Desarrollo y Requisitos

Para la replicación del entorno de desarrollo utilizado por **Lucía Fernández Florencio**, se requieren las siguientes especificaciones:

11.2.1. Requisitos de Software

- **Entorno de Ejecución:** Node.js (v18.0.0 o superior).
- **Gestor de Paquetes:** npm o yarn.

- **IDE Recomendado:** Visual Studio Code con extensiones para ESLint y Tailwind CSS.
- **Control de Versiones:** Git v2.30+.

11.2.2. Configuración Inicial (Setup)

1. **Clonación del Repositorio:** git clone <url-repositorio>
2. **Instalación de Dependencias:** npm install

Variables de Entorno: Es imperativo configurar un archivo .env en la raíz con las siguientes claves privadas:
Fragmento de código
VITE_SUPABASE_URL
VITE_SUPABASE_ANON_KEY

11.3. Diccionario de Datos y Modelo Relacional

La base de datos PostgreSQL en Supabase se fundamenta en tres entidades núcleo que garantizan la trazabilidad financiera.

11.3.1. Especificación de Tablas Principales

Tabla	Columna	Tipo	Descripción
profiles	id	uuid	PK vinculada a auth.users.
	email	text	Correo electrónico del autónomo.

	role	user_role	Rol definido en la matriz de permisos.
projects	id	uuid	Identificador único del proyecto.
	user_id	uuid	FK hacia profiles.
	status	project_status	Estado (active, archived, etc.).
transactions	id	uuid	Registro atómico de ingreso o gasto.
	project_id	uuid	FK vinculada a la unidad de negocio.
	amount	numeric	Importe neto con precisión decimal.

11.4. Seguridad y Row Level Security (RLS)

Siguiendo el principio de **Mínimo Privilegio**, se han implementado políticas de seguridad a nivel de fila que impiden que un autónomo acceda a los datos de otro.

Ejemplo de Política RLS para Transacciones:

```
CREATE POLICY "Users can only see their own transactions"
ON transactions FOR SELECT
USING (auth.uid() = user_id);
```

Esta capa de seguridad garantiza que la API de Supabase sólo devuelva registros cuyo user_id coincida con el token JWT del usuario autenticado.

11.5. Mantenimiento y Resolución de Problemas (Troubleshooting)

Para el cierre del proyecto el **30 de enero**, se han identificado los siguientes escenarios de mantenimiento:

11.5.1. Sincronización de Datos

Si los balances del dashboard no coinciden con los registros, se debe verificar el estado del *hook* useTaxLogic. El sistema utiliza una estrategia de caché **SWR**, por lo que un refresco forzado de la caché del navegador suele resolver discrepancias menores.

11.5.2. Límites de Almacenamiento

AutonoStack opera bajo el *Free Tier* de Supabase. Si se alcanza el límite de **1GB** en el Bucket de justificantes, el sistema devolverá un error 413 (Payload Too Large). Se recomienda la compresión previa de PDFs o el escalado al plan Pro.

11.6. Interfaz de Programación de Aplicaciones (API) y Servicios Externos

La arquitectura de **AutonoStack** no depende de un backend monolítico, sino que consume servicios distribuidos a través de APIs de alto rendimiento que garantizan la soberanía de los datos.

11.6.1. Integración con Supabase API (PostgREST)

El acceso a la base de datos PostgreSQL se realiza mediante el SDK de Supabase, que expone una interfaz **RESTful** automática basada en el esquema de la base de datos.

- **Protocolo de Comunicación**: Todas las peticiones se realizan bajo el protocolo seguro **HTTPS**.
- **Autenticación de Peticiones**: Cada llamada a la API incluye un token **JWT (JSON Web Token)** en las cabeceras de autorización, validado en el servidor para aplicar las políticas **RLS**.
- **Endpoints Críticos**:
  - /auth/v1: Gestión de registros, inicios de sesión y recuperación de contraseñas.
  - /rest/v1/transactions: Acceso atómico al libro diario de ingresos y gastos.
  - /storage/v1: Gestión de subida y descarga de justificantes legales en formato PDF o imagen.

11.6.2. Servicios de Generación de Informes (jsPDF)

Para cumplir con el hito de los manuales del 21 de noviembre, se integró el servicio de generación de documentos en el cliente.

- **Funcionalidad**: Transforma los datos JSON de la API en documentos **PDF** estructurados.
- **Lógica de Negocio**: Calcula el **Margen de Beneficio** en tiempo de ejecución para incluirlo en el reporte.

11.7. Guía de Despliegue y Configuración en la Nube (Vercel)

El despliegue final de **AutonoStack** se consolidó el **09 de enero**, tras superar las pruebas en entorno local. Se ha optado por **Vercel** debido a su integración nativa con el *bundler* **Vite** y su red de nodos *Edge* de baja latencia.

11.7.1. Procedimiento de Despliegue (Pipeline CI/CD)

El flujo de puesta en producción sigue un protocolo de integración continua automatizado:

1. **Vinculación de Repositorio**: Conexión del repositorio de **GitHub** con el panel de control de Vercel.
2. **Detección de Framework**: El motor de Vercel identifica automáticamente la configuración de **Vite** y el punto de entrada de la **SPA**.
3. **Inyección de Secretos**: Configuración manual de las variables de entorno en el panel de Vercel para evitar la exposición de claves en el código fuente.
4. **Build & Edge Deployment**: Compilación del código TypeScript y distribución de los activos estáticos (HTML/JS/CSS) en la red global de nodos.

11.7.2. Variables de Entorno en Producción

Para el correcto funcionamiento en el dominio final, el entorno debe contener las siguientes claves configuradas:

Variable	**Descripción**	**Valor / Origen**
VITE_SUPABASE_URL	Endpoint del proyecto Supabase.	Panel de Supabase Settings.
VITE_SUPABASE_ANON_KEY	Clave pública de acceso a la API.	Panel de API de Supabase.
NODE_ENV	Entorno de ejecución.	Establecido en production.

11.8. Protocolos de Mantenimiento y Cierre de Proyecto

Siguiendo el cronograma, tras el hito de despliegue del 09 de enero, se inició la fase de optimización final.

- **16 de enero (Ajustes)**: Auditoría de rendimiento mediante **Lighthouse** para asegurar que el aplicativo cumple con el requisito de carga instantánea.
- **23 de enero (Documentación)**: Cierre definitivo de los manuales técnicos, asegurando que las rutas de la API y las tablas del diccionario de datos coinciden con la realidad del sistema.
- **30 de enero (Empaquetado)**: Generación del *bundle* final y preparación del entorno para la defensa del proyecto.

11.9. Organización del Código Fuente y Modularidad

Siguiendo las mejores prácticas de **Clean Architecture**, el código se ha estructurado de forma que la lógica de negocio (cálculos fiscales) esté desacoplada de la lógica de interfaz (componentes visuales).

11.9.1. Árbol de Directorios (/src)

Directorio	Responsabilidad Técnica	Contenido Relevante
src/components	Componentes de UI atómicos y modulares.	Dashboard, TransactionTable, ProjectCard.

src/hooks	Lógica de estado y llamadas asíncronas.	useTaxLogic.ts, useAuth.ts, useProjects.ts.
src/lib	Configuración de servicios externos.	supabaseClient.ts (Instancia de la API).
src/types	Definiciones estáticas de **TypeScript**.	Interfaces para Transaction, Project y Profile.
src/utils	Funciones de ayuda y formateo.	formatCurrency.ts, dateUtils.ts.
src/styles	Configuración global de estilos.	index.css (Directivas de **Tailwind CSS**).

11.10. Lógica de Componentes y Hooks Personalizados

El núcleo de la inteligencia de **AutonoStack** reside en sus *Custom Hooks*, que actúan como puentes entre la base de datos de **Supabase** y la interfaz de **React**.

11.10.1. El Hook Maestro: useTaxLogic

Este componente es el encargado de procesar la información del hito del **05 de diciembre** (Dashboard Analítico). Realiza las siguientes operaciones en tiempo real:

1. **Filtrado Asíncrono**: Recupera transacciones mediante el SDK de Supabase.
2. **Cálculo de Margen**: Aplica la fórmula de rentabilidad para cada unidad de negocio.
3. **Previsión de IVA**: Acumula el IVA repercutido y soportado para ofrecer la estimación del modelo 303 de la AEAT.

11.11. Procedimientos de Pruebas y Aseguramiento de Calidad (QA)

Como se detalló en el cronograma, el periodo entre el **19 de diciembre** (Testing Local) y el **16 de enero** (Ajustes Finales) se dedicó íntegramente a la validación del sistema.

11.11.1. Niveles de Pruebas Ejecutadas

- **Pruebas de Integración**: Validación de la comunicación entre los formularios de React y los *Buckets* de **Supabase Storage** para asegurar que los PDFs se almacenan correctamente.
- **Pruebas de Entorno (TomCat vs Vercel)**: Verificación de que las variables de entorno se inyectan correctamente en el hito del **09 de enero** sin exponer las claves privadas.
- **Pruebas de Rendimiento**: Uso de *Lighthouse* para garantizar que el aplicativo cumple con los requisitos de carga rápida en dispositivos móviles.

11.12. Gestión de Dependencias y Seguridad de Paquetes

El archivo package.json ha sido auditado para evitar vulnerabilidades, utilizando **TypeScript** para minimizar los errores en tiempo de ejecución.

Librería	Versión	Función Crítica
React	18.2.0	Motor de renderizado de la SPA.
@supabase/supabase-js	2.x.x	Cliente oficial para la gestión de Auth y DB.
Tailwind CSS	3.x.x	Framework de diseño atómico.
Tremor / Recharts	3.x.x	Librerías de visualización de datos financieros.
jsPDF	2.5.x	Generador de informes PDF definido el 21 de nov.

11.13. Consideraciones Finales de Ingeniería

El empaquetado final del software, realizado el **30 de enero**, garantiza que el código sea "isomórfico"; es decir, que pueda ser desplegado en cualquier plataforma compatible con Node.js, aunque Vercel sea la recomendada por su optimización de costes y rendimiento.

Nota para Auditores: Las políticas de **Row Level Security (RLS)** son la última línea de defensa. Cualquier intento de acceso a la API sin un token JWT válido resultará en un error *401 Unauthorized* a nivel de base de datos, independientemente del estado del frontend.

12. Conclusiones y Valoración del Proyecto

El desarrollo de **AutonoStack** ha representado un viaje de dieciocho semanas desde la conceptualización de una idea hasta la puesta en producción de un ecosistema Full Stack. Al alcanzar el hito final del **30 de enero**, se han extraído las siguientes conclusiones fundamentales:

12.1. Cumplimiento de Objetivos y Hitos

- **Soberanía Financiera:** Se ha logrado transformar el registro contable tradicional en una herramienta de análisis estratégico, cumpliendo con la visión definida el 12 de septiembre.
- **Solvencia Técnica:** La integración de **React** y **Supabase** ha permitido alcanzar un rendimiento superior, con tiempos de carga inferiores a 2 segundos (**RNF-02**).
- **Seguridad Garantizada:** La implementación de **RLS** ha blindado la información de **bruma.studio**, asegurando que la privacidad no sea una opción, sino una característica intrínseca del sistema.

12.2. Valoración Personal y Aprendizaje

Este proyecto ha permitido consolidar habilidades en la gestión del ciclo de vida completo del software (SDLC). La mayor complejidad no residió en la codificación, sino en la **ingeniería de requisitos** y en asegurar que cada hito del cronograma aportara un valor incremental real al producto final.

13. Trabajo Futuro y Escalabilidad

Aunque el proyecto se encuentra en un estado de **"Completado"** para su defensa académica el 30 de enero, la arquitectura se ha diseñado para ser escalable, permitiendo la implementación de las fases pactadas en la hoja de ruta estratégica:

- **Fase 2: Automatización Bancaria:** Integración de APIs financieras para la conciliación automática de movimientos, eliminando el volcado manual.
- **Fase 3: Inteligencia Predictiva:** Implementación de modelos de Machine Learning para la categorización automática de gastos basados en patrones históricos.
- **Fase 4: Multi-perfil:** Evolución hacia un modelo de estudio creativo colaborativo, permitiendo la gestión de gastos compartidos entre varios autónomos.

14. Bibliografía y Recursos Utilizados

Para la elaboración de este proyecto y su documentación técnica, se han consultado las siguientes fuentes y herramientas:

1. **Documentación Oficial de React:** Guías de referencia para la arquitectura de componentes y Hooks.
2. **Supabase Documentation:** Especificaciones de PostgreSQL, Auth y Row Level Security.
3. **Tailwind CSS Documentation:** Diccionario de clases de utilidad para el diseño atómico.
4. **W3C WCAG 2.1:** Pautas de accesibilidad para el cumplimiento del nivel AA.
5. **Vercel Deployment Guide:** Protocolos de despliegue en la red Edge y gestión de variables de entorno.

15. Anexos

Anexo I: Matriz de Trazabilidad de Requisitos

La **MTR** constituye el mecanismo de control de calidad que vincula el origen de cada necesidad con su correspondiente requisito funcional y su validación final. Esta herramienta garantiza que el desarrollo no se desvíe de los objetivos estratégicos y que cada línea de código responda a una demanda real de **bruma.studio**.

Estructura y Validación Técnica

Para una gestión eficiente, la trazabilidad se organiza bajo los siguientes criterios de verificación:

ID	Origen / Fuente	Descripción del Requisito	Prioridad	Actor Responsable	Método de Verificación
RF-01	Acta de Inicio	Autenticación y gestión de sesiones	Alta	Autónomo / Admin	Prueba de login y

		mediante Supabase Auth.			expiración de token.
RF-02	Propuesta 1.2	Operaciones CRUD para facturas emitidas y recibidas.	Alta	Autónomo	Inspección de registros en PostgreSQL.
RF-03	Acta de Inicio	Categorización de gastos con adjunto de justificantes.	Media	Autónomo	Prueba de subida de archivos (Storage).
RF-04	Propuesta 3.1	Asociación de transacciones a proyectos para margen neto.	Alta	Autónomo	Verificación de integridad referencial SQL.
RF-05	Propuesta 3.2	Dashboard con gráficas y previsión de IVA trimestral.	Alta	Autónomo	Validación de cálculos en componentes UI.
RF-08	Propuesta 4.1	Acceso a "Modo Demo" con datos ficticios sin registro.	Baja	Invitado	Acceso por ruta pública sin sesión activa.

RNF-01	Propuesta 4.2	Seguridad a nivel de fila (RLS) para aislamiento de datos.	Crítica	Administrador	Intento de acceso a registros ajenos vía API.
RNF-03	Estándar WCAG	Cumplimiento de accesibilidad Nivel AA.	Crítica	Todos	Auditoría con Lighthouse / Axe.
RNF-04	Propuesta 4.2	Interfaz responsive (Móvil / Escritorio).	Media	Todos	Prueba de visualización en diferentes viewports.

Justificación Estratégica de la Trazabilidad

- **Vínculo con la Consultoría:** Asegura que puntos críticos discutidos en el Acta **REF: AR-2025-001**, como la necesidad de prever el IVA trimestral, se transformen efectivamente en el **RF-05**.
- **Gestión Integral de Cambios:** Permite identificar rápidamente qué validaciones se ven afectadas ante cualquier modificación en la asociación de gastos a proyectos, manteniendo la integridad del sistema ante evoluciones futuras.
- **Compromiso de Accesibilidad:** Se incluye una traza específica para garantizar que el compromiso **WCAG AA** se aplique transversalmente sobre todos los requisitos, especialmente en la visualización de datos complejos.

Anexo II: Glosario Técnico

- **BaaS (Backend as a Service):** Modelo de servicios en la nube (como **Supabase**) que permite delegar la gestión de bases de datos, autenticación y almacenamiento para centrarse en la lógica del cliente.
- **Edge Network:** Red global de servidores de proximidad utilizada por **Vercel** para servir el contenido de la aplicación con la mínima latencia posible.
- **MVP (Mínimo Producto Viable):** Versión inicial de la plataforma que incluye las funcionalidades críticas (CRUD, Dashboard, Seguridad) necesarias para satisfacer las necesidades de **bruma.studio**.
- **React.js:** Biblioteca de JavaScript basada en componentes y un **Virtual DOM** eficiente para construir interfaces de usuario reactivas y dinámicas.
- **SPA (Single Page Application):** Arquitectura web donde la aplicación se carga una sola vez y las interacciones posteriores no requieren recargar la página completa.
- **JWT (JSON Web Token):** Estándar de seguridad para representar notificaciones entre dos partes de forma segura, utilizado en **AutonoStack** para gestionar sesiones de usuario.
- **PostgreSQL:** Sistema de gestión de bases de datos relacional de código abierto que garantiza la integridad atómica de los datos financieros.
- **RLS (Row Level Security):** Capa de seguridad en la base de datos que restringe el acceso a las filas de una tabla basándose en la identidad del usuario que realiza la consulta.
- **TypeScript:** Superconjunto de JavaScript que añade tipado estático, permitiendo detectar errores durante el desarrollo y asegurar que los datos financieros tengan la estructura correcta.
- **AEAT (Agencia Estatal de Administración Tributaria):** Organismo público encargado de la gestión del sistema tributario en España, cuyos modelos (como el 303) se previsualizan en el dashboard.

- **IVA (Impuesto sobre el Valor Añadido):** Tributo indirecto que grava el consumo; **AutonoStack** calcula tanto el **Repercutido** (ventas) como el **Soportado** (compras).
- **Margen Neto por Proyecto:** Indicador de rentabilidad estratégica.
- **Tremor / Recharts:** Ecosistema de librerías de visualización utilizadas para transformar registros contables en gráficas de flujo de caja y KPIs accionables.
- **CI/CD (Integración y Despliegue Continuo):** Automatización del ciclo de vida del software que permite desplegar cambios en **Vercel** de forma segura tras cada revisión de código.
- **Sprint:** Ciclo de trabajo de dos semanas dentro de la metodología **Scrum** para la entrega de incrementos funcionales de software.
- **WCAG 2.1 (Nivel AA):** Estándares internacionales de accesibilidad web que aseguran que la plataforma sea utilizable por personas con diversas capacidades.